

AGRO OPS

Plan Integral v4.1

Arquitectura técnica, desarrollo incremental, integración con Finnegans y ARCA, operación por dominios y estrategia de pruebas

OBJETIVO CENTRAL

Construir una aplicación operativa agropecuaria propia que sea la fuente de verdad productiva de la empresa, manteniendo Finnegans como fuente de verdad contable/fiscal y ARCA como fuente legal de Carta de Porte Electrónica. El sistema se desarrolla por etapas, con una arquitectura probada desde el inicio y con pruebas automáticas de regresión en cada push a GitHub.

ACTUALIZACIÓN v4.1

Se mantiene íntegramente el alcance y todas las tareas del Plan Integral v4.0. Se incorpora una nueva **Etap 1.5 - Diseño de datos y DER maestro conceptual**, ubicada entre el Walking Skeleton y el Núcleo transversal. La numeración posterior se conserva para no alterar referencias existentes.

Componente	Decisión
Frontend	Next.js + React + TypeScript + MapLibre; PWA online-first; deploy en Vercel.
Backend	Rust + Axum + Tokio + SQLx; API, worker persistente y jobs programados desde una misma codebase e imagen Docker.
Datos	PostgreSQL + PostGIS; Supabase Auth y Storage en staging/producción.
Ingeniería	GitHub, Docker y GitHub Actions como mecanismos obligatorios de reproducibilidad y calidad.

Índice

1. Resumen ejecutivo y cambio de arquitectura
2. Principios no negociables
3. Arquitectura objetivo y responsabilidades
4. Arquitectura de desarrollo, entornos y deployment
5. GitHub, CI/CD y sistema de pruebas
6. Estrategia incremental y Definition of Done
7. Etapa 0 - Repositorio y gobierno técnico
8. Etapa 1 - Walking Skeleton
- 8A. Etapa 1.5 - Diseño de datos y DER maestro conceptual
9. Etapa 2 - Núcleo transversal
10. Etapa 3 - Territorio, campañas y cartografía
11. Etapa 4 - Stock operativo
12. Etapa 5 - Agricultura
13. Etapa 6 - Maquinaria y vehículos
14. Etapa 7 - Ganadería
15. Etapa 8 - Granos
16. Etapa 9 - Comercialización de granos
17. Etapa 10 - Integración ARCA/CPE
18. Etapa 11 - Integración Finnegans
19. Etapa 12 - Costos, resultados y Margen Bruto
20. Etapa 13 - Reportes, conciliación y alertas
21. Etapa 14 - Piloto, migración y cutover
22. Flujos operativos end-to-end
23. Matriz de pruebas y precisión esperada
24. ADR, cambios de arquitectura y evolución futura
25. Decisiones pendientes y POC
26. Roadmap resumido y orden de ejecución

1. Resumen ejecutivo y cambio de arquitectura

El Plan Integral v3.1 separaba Contabilidad y Gestión Agropecuaria en dos empresas lógicas dentro del ecosistema Finnegans. La versión 4.1 mantiene la misma separación conceptual, pero cambia el lugar donde vive la operación: Agro Ops pasa a ser el sistema operativo productivo y Finnegans queda concentrado en registración financiera, fiscal, tesorería, impuestos y contabilidad.

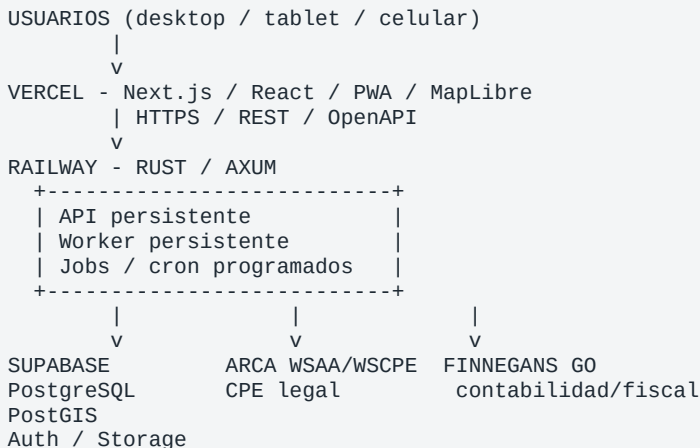
DECISIÓN DE ARQUITECTURA

ARCA conserva la verdad legal de la CPE; Agro Ops conserva la verdad operativa y productiva; Finnegans conserva la verdad contable/fiscal. Las integraciones transfieren solamente la consecuencia que el otro sistema necesita, con trazabilidad e idempotencia.

1.1 Por qué se cambia el enfoque

- Evitar forzar la operación agropecuaria dentro de estructuras genéricas de ERP cuando el negocio requiere lotes variables por campaña, cartografía, relaciones flexibles entre documentos y flujos operativos propios.
- Conservar Finnegans para aquello donde el ERP aporta valor y donde existen obligaciones legales/fiscales específicas.
- Diseñar stock, agricultura, granos, ganadería, maquinaria y resultados con reglas propias, auditables y testeables.
- Poder desarrollar y validar cada módulo sin depender de que toda la migración esté terminada.
- Mantener la posibilidad de modificar el diseño funcional con bajo costo durante las primeras etapas.

1.2 Arquitectura objetivo



1.3 Alcance funcional que se conserva

Dominio	Alcance
Territorio	Establecimientos, lote base geográfico, campañas, unidades operativas variables, cartografía clickeable.
Agricultura	Planificación, labores, insumos, cosecha, producción y contexto por lote/campaña.
Stock	Insumos y granos mediante ledger de movimientos, depósitos, transferencias, ajustes y reversión.
Ganadería	Existencias, categorías/rodeos, movimientos, pesadas, cambios de categoría, compras/ventas y costos.
Maquinaria/Vehículos	Equipos, horas/km, combustible, mantenimiento, reparaciones, costos y asignación productiva.
Granos/Comercial	Contratos, CPE, certificados, liquidaciones, fletes, kilos comprometidos/liquidados y trazabilidad.
Resultados	Costos reales, costos internos, resultados productivos y Margen Bruto.

Integraciones	ARCA y Finnegans con outbox, jobs, reintentos, idempotencia, conciliación y auditoría.
---------------	--

2. Principios no negociables

Principio	Regla
Fuente de verdad explícita	Cada dato debe tener un sistema dueño. No se admiten ediciones paralelas sin regla de propiedad.
Carga manual única	Un evento real se carga manualmente una sola vez; las demás representaciones se sincronizan o derivan.
Contabilidad fuera de Agro Ops	Agro Ops no reemplaza la registración fiscal, tributaria, bancaria ni contable de Finnegans.
ARCA como autoridad legal de CPE	Agro Ops replica y relaciona la CPE, pero no pretende reemplazar la fuente legal.
Monolito modular	Una única codebase Rust con límites internos claros. No microservicios en V1.
API/worker/jobs separados por proceso	La API atiende usuarios; el worker procesa integración y outbox; los jobs programados ejecutan tareas periódicas.
Transacciones e idempotencia	Los cambios de negocio, audit y outbox se confirman juntos; los reintentos no duplican efectos.
No borrado destructivo	Las operaciones confirmadas se anulan/revierten; no se eliminan físicamente.
Precisión numérica	Dinero, kg, litros, hectáreas, horas y km utilizan Decimal; no f64 como dato crítico.
Docker reproducible	El stack del proyecto se define en contenedores para evitar dependencias globales inconsistentes.
GitHub como fuente del código	Todo cambio pasa por branch, commit, push, CI y PR; main protegida.
Tests antes del merge	Una feature no está terminada si no tiene pruebas que demuestren sus invariantes y que no rompe lo anterior.

3. Arquitectura objetivo y responsabilidades

3.1 Fuente de verdad por tipo de información

Información	Sistema dueño	Tratamiento en los otros sistemas
CPE/CTG legal	ARCA	Agro Ops replica, normaliza y relaciona; Finnegans recibe/usa lo que requiera su circuito.
Establecimientos/lotos/campañas/UOP	Agro Ops	Finnegans solo recibe dimensiones/referencias si son necesarias para contabilización o análisis.
Labores, consumos, cosecha	Agro Ops	Finnegans recibe solo consecuencias reales necesarias: costos/stock/documentos que correspondan.
Stock operativo	Agro Ops	Se concilia con la representación patrimonial definida en Finnegans.
Hacienda operativa	Agro Ops	Compras/ventas fiscales permanecen en Finnegans; movimiento físico se refleja en Agro Ops.
Maquinaria y uso	Agro Ops	Facturas/costos reales de terceros nacen en Finnegans y se asignan operativamente en Agro Ops.
Contratos operativos	Agro Ops	Finnegans puede conservar la representación necesaria para liquidaciones/contabilidad según circuito validado.
Facturas, IVA, retenciones, pagos	Finnegans	Agro Ops recibe referencias/costos cuando impactan resultado.
Liquidación fiscal de granos	Finnegans	Agro Ops recibe kilos, precio, gastos y vínculos;

		no vuelve a emitir el documento fiscal.
Margen Bruto/resultado productivo	Agro Ops	No se sincroniza como contabilidad ficticia.

3.2 Componentes técnicos

Componente	Responsabilidad	Tecnología
Web/PWA	Interfaz, formularios, mapas, dashboards, validaciones UX no autoritativas.	Next.js, React, TypeScript, MapLibre, Vercel.
API	Casos de uso sincrónicos, autenticación/autorización, reglas de negocio, transacciones.	Rust, Axum, Tokio, SQLx.
Worker	Outbox, integración ARCA/Finnegans, retries, procesamiento asíncrono.	Misma codebase Rust; proceso persistente separado.
Jobs/Cron	Sincronizaciones periódicas, conciliaciones, limpieza/alertas programadas.	Misma imagen Rust ejecutada con comando job específico.
DB	Datos operativos, ledger, relaciones, audit, outbox, jobs.	PostgreSQL; PostGIS para geometrías.
Auth	Identidad y sesión.	Supabase Auth; autorización efectiva en Rust.
Storage	PDF/adjuntos/KML/GeoJSON/documentación.	Supabase Storage; buckets privados.
CI/CD	Validación automática y deployment.	GitHub Actions + Vercel/Railway.

3.3 Estructura del monolito modular

```

services/backend/
  src/
    api/
    worker/
    jobs/
  crates/
    domain/
    application/
    infrastructure/
    territory/
    inventory/
    agriculture/
    machinery/
    livestock/
    grain/
    commercial/
    costs/
    arca/
    finnegans/
    common/

```

REGLA DE DEPENDENCIA

El dominio no conoce Axum, Supabase, SOAP, ARCA ni Finnegan. Las integraciones quedan detrás de adapters. La API coordina casos de uso; no contiene lógica de negocio compleja.

4. Arquitectura de desarrollo, entornos y deployment

4.1 Desarrollo local en contenedores

El objetivo es que una PC nueva pueda clonar el repositorio y levantar un entorno coherente sin instalar globalmente todo el stack. La máquina anfitriona necesita principalmente Git, Docker/Docker Compose y el editor/Codex.

```

docker compose up

web-dev      -> Node / pnpm / Next.js

```

```
api-dev      -> Rust / Cargo / Axum / SQLx CLI / hot reload
worker-dev   -> misma imagen Rust, comando worker
postgres-dev -> PostgreSQL + PostGIS
```

4.2 Entornos

Entorno	Datos	Aplicación	Uso
Local	PostgreSQL/PostGIS en Docker.	Frontend, API y worker locales en Docker.	Desarrollo rápido, tests y migrations.
Staging	Proyecto Supabase independiente.	Vercel staging/preview + Railway staging.	Integración real con Auth/Storage y POC externos.
Production	Supabase Production independiente.	Vercel Production + Railway Production.	Operación real posterior al cutover.

4.3 Imagen Docker Rust

Se utilizará un Dockerfile multi-stage: target de desarrollo, etapa de compilación y runtime final pequeño. API, worker y cron jobs reutilizan la misma imagen y cambian solamente el comando de arranque.

```
agro-ops:<version>

Railway service agro-api      -> agro-ops api
Railway service agro-worker   -> agro-ops worker
Railway cron sync-arca        -> agro-ops job sync-arca
Railway cron reconcile         -> agro-ops job reconcile
```

4.4 Variables y secretos

- Los secretos nunca se commitean a GitHub.
- ARCA certificate/private key, tokens y credenciales Finnegans se guardan en secrets de infraestructura por ambiente.
- La aplicación recibe configuración mediante variables de entorno tipadas y validadas al inicio.
- Producción y staging no comparten claves, bases ni buckets.

5. GitHub, CI/CD y sistema de pruebas

5.1 Flujo obligatorio de cambios

```
main (protegida)
|
+-- feat/territory-base-plots
+-- feat/inventory-ledger
+-- feat/arca-import
+-- fix/grain-reversal

branch -> desarrollo -> tests locales -> commit -> push -> GitHub Actions -> PR -> review -> merge
```

REGLA DE MERGE

Si un required check falla, el PR no se mergea. La suite de pruebas funciona como barrera automática contra regresiones y contra implementaciones de Codex que no respeten contratos previos.

5.2 Pipeline en cada push

Bloque	Checks mínimos
Rust estático	cargo fmt --check; cargo clippy --all-targets --all-features -- -D warnings; cargo check.

Rust unit/domain	cargo test para dominio, casos de uso, invariantes y property-based tests rápidos.
Database	Levantar PostgreSQL/PostGIS, aplicar migrations desde cero, constraints, integration tests SQLx/PostGIS.
API	Tests de endpoints + estado DB + audit + outbox + idempotencia cuando corresponda.
Frontend	lint, TypeScript typecheck, unit/component tests y build.
Docker	Build completo de la imagen runtime y smoke test de arranque.
E2E crítico	Flujos esenciales con navegador y stack real de test; se amplía progresivamente con cada módulo.

5.3 Niveles de prueba

Nivel	Qué prueba	Precisión esperada
Unitario	Funciones puras, value objects, validaciones e invariantes.	Debe indicar exactamente qué regla falló; casos límite explícitos.
Property-based	Conservación, reversión, cantidades y combinaciones generadas.	Cientos/miles de inputs para reglas críticas.
DB integration	SQL real, constraints, transacciones, locking, PostGIS.	No mockear comportamiento que depende de PostgreSQL.
API integration	Request -> caso de uso -> DB/outbox/audit.	No alcanza con status HTTP: comprobar side effects exactos.
Contract/adapters	ARCA/Finnegans con fixtures y mocks.	Payloads exactos, errores, timeout, retry, idempotencia.
E2E	Flujo visible completo.	Validar el camino crítico del usuario y los datos resultantes.
Mutation	Sensibilidad real de tests críticos.	Nightly/milestone en stock, granos, costos e integraciones.
Nightly externo	Homologación ARCA/Finnegans staging cuando aplique.	Detectar cambios de contratos externos sin romper CI de cada push.

5.4 Filosofía de precisión

- Un test no debe limitarse a "devuelve 200" cuando existe una consecuencia operativa.
- Si una operación consume stock, se verifica movimiento, saldo, entidad origen, audit y outbox.
- Si una operación se anula, se demuestra que la reversión deja las cantidades en el estado esperado y preserva historia.
- Si un evento puede repetirse por red o retry, se prueba explícitamente la idempotencia.
- Si existe concurrencia posible, se prueba que dos requests simultáneos no generan stock negativo o doble imputación.
- Fixtures deben tener intención: nombre de caso, expected state y explicación de por qué es válido.

6. Estrategia incremental y Definition of Done

El sistema se construye como un walking skeleton que funciona desde el inicio. Cada etapa añade un conjunto pequeño de capacidades y deja el producto ejecutable. No se avanza a una gran vertical funcional hasta que la infraestructura de pruebas, auditoría y deployment de esa vertical esté lista.

6.1 Definition of Done de una feature

Criterio	Obligatorio
Arquitectura respetada	Sí
Modelo de dominio y invariantes explícitas	Sí
Migration versionada si cambia schema	Sí
API contract documentado	Sí
Validaciones y permisos	Sí

Unit/invariant tests	Sí
DB integration tests	Sí
API tests	Sí
Audit	Cuando corresponda
Idempotencia	Cuando corresponda
Outbox/worker	Cuando corresponda
Frontend tests	Sí
E2E del flujo crítico	Sí
Docker build	Sí
CI GitHub verde	Sí
Staging validado	Sí
Documentación/ADR actualizada	Sí

REGLA DE PROGRESO

La pantalla funcionando no significa feature terminada. El criterio de salida es comportamiento correcto, persistencia correcta, integración controlada, pruebas precisas y CI verde.

7. Etapa 0 - Repositorio y gobierno técnico

Crear la base de ingeniería antes de escribir lógica agropecuaria. El objetivo es que el proyecto tenga una estructura estable, control de versiones, reglas de arquitectura y documentación de decisiones.

Flujo operativo esperado en Agro Ops

- Crear repositorio GitHub agro-ops y estructura de monorepo.
- Configurar main protegida, convenciones de branches y PR template.
- Crear docs/adr y registrar las decisiones iniciales: Rust/Axum, monolito modular, PostgreSQL/PostGIS, Docker, Supabase, Vercel, Railway, ledger, outbox, ARCA read-only V1.
- Definir CODEOWNERS/review si más de una persona participa y reglas para Codex: no modificar arquitectura sin ADR.
- Crear README con comandos reproducibles de desarrollo, test y build.

Rol de Finnegan y flujo entre sistemas

- No existe integración operativa con Finnegan en esta etapa.
- Se documenta desde el inicio que Finnegan es sistema contable/fiscal y que el adapter será el único punto de acoplamiento.
- Se deja explícita la restricción del proyecto: no depender de BProc para resolver la integración.

Pruebas obligatorias de la etapa

- CI mínimo que valide estructura, formatting y build vacío.
- Test de que main requiere checks antes del merge.
- Linter/arquitectura para impedir dependencias no permitidas cuando se incorporen crates.

CRITERIO DE SALIDA

Repositorio clonable, reglas Git activas, ADR iniciales aprobados y primer PR mergeado por el flujo completo.

8. Etapa 1 - Walking Skeleton

Demostrar la arquitectura completa de punta a punta antes de construir operación. El usuario debe poder abrir una web, autenticarse en staging, consultar el backend Rust y comprobar que la base, worker y versión desplegada están saludables.

Flujo operativo esperado en Agro Ops

- Levantar docker compose con web-dev, api-dev, worker-dev y PostgreSQL/PostGIS.

Flujo operativo esperado en Agro Ops (continuación de Etapa 1)

- Implementar GET /health, /ready y /version.
- Crear página System Status en Next.js.
- Desplegar frontend en Vercel, API y worker en Railway, DB staging en Supabase.
- Configurar logging/tracing con request_id y correlation_id.
- Agregar OpenAPI generado desde el backend.

Rol de Finnegans y flujo entre sistemas

- Finnegans todavía no recibe ni entrega datos; solo se prepara el módulo integrations/finnegans.
- El diseño del adapter impide que entidades de negocio dependan de IDs internos de Finnegans.
- Se documenta cómo se probará la conectividad en un POC posterior.

Pruebas obligatorias de la etapa

- Test de health/ready con DB disponible y caída.
- Test de migrations desde base vacía.
- Test de worker heartbeat y toma de job de prueba.
- Smoke test Docker de la imagen de producción.
- E2E: abrir web -> autenticar -> system status -> API/DB/worker OK.

CRITERIO DE SALIDA

Un entorno staging real muestra la aplicación, API y worker desplegados y todos los checks de GitHub pasan automáticamente.

8A. Etapa 1.5 - Diseño de datos y DER maestro conceptual

Formalizar el modelo de información antes de materializar el núcleo y las verticales de negocio. El objetivo es acordar entidades, relaciones, propiedad de datos, historia e invariantes sin convertir prematuramente todo el diseño conceptual en tablas físicas.

Flujo operativo esperado en Agro Ops

- Crear y versionar **docs/data-model/conceptual-model.md** y **docs/data-model/agro-ops-erd.mmd** como fuentes editables del modelo.
- Inventariar las entidades principales por dominio y declarar el sistema dueño de cada dato.
- Definir cardinalidades, obligatoriedad/opcionalidad y sentido de cada relación relevante.
- Separar explícitamente el **DER maestro conceptual** del schema físico PostgreSQL; cada etapa posterior materializa solo las tablas que necesita.

Flujo operativo esperado en Agro Ops (continuación)

- Definir estrategia de IDs internos UUID, códigos canónicos estables y referencias a sistemas externos.
- Definir ciclo de vida e historia: qué entidades son mutables, cuáles se versionan y cuáles se anulan/revierten en lugar de borrarse.
- Definir entidades geoespaciales, tipos PostGIS y SRID; documentar cómo lote base y unidades operativas preservan historia entre campañas.
- Definir tipos críticos: Decimal para dinero/kg/litros/ha/horas/km, timestamps, estados y metadatos de auditoría.
- Validar el DER contra los flujos end-to-end ya previstos: territorio, stock, agricultura, maquinaria, ganadería, granos y comercialización.
- Asignar a cada entidad la etapa en la que se materializará mediante migrations, evitando crear tablas “por adelantado” sin caso de uso.

Rol de Finnegans, ARCA y flujo entre sistemas

- Finnegans y ARCA se modelan como sistemas externos; el dominio de Agro Ops no incorpora sus IDs internos como claves primarias ni dependencias estructurales.
- Las relaciones externas se representan mediante ExternalReference y claves canónicas; CPE conserva a ARCA como autoridad legal.
- El DER debe mostrar qué consecuencia cruza cada frontera de sistema sin duplicar la propiedad del dato.

Pruebas y validaciones obligatorias de la etapa

- El archivo Mermaid del DER renderiza correctamente y su sintaxis se valida en CI cuando se incorpore el check documental.
- Cada entidad tiene owner, identificador, ciclo de vida y etapa de implementación definidos.
- Cada relación crítica declara cardinalidad y nulabilidad; no quedan vínculos ambiguos por descripción libre.
- Caso territorial: campaña 1 y campaña 2 pueden subdividir el mismo lote base de forma distinta sin alterar historia previa.
- Caso comercial: contrato, CPE, certificado y liquidación pueden relacionarse sin exigir un orden temporal rígido.
- No existe dependencia del dominio hacia IDs internos de Finnegans/ARCA; toda integración queda detrás de referencias/adapters.
- Revisión explícita del modelo antes de iniciar migrations de dominio del Núcleo transversal y etapas posteriores.

CRITERIO DE SALIDA

DER maestro conceptual versionado y aprobado, con dominios, entidades, cardinalidades, invariantes, propiedad del dato, reglas de historia y mapa de implementación por etapa. Las migrations posteriores pueden apoyarse en este modelo sin convertirlo en un schema monolítico anticipado.

9. Etapa 2 - Núcleo transversal

Construir las capacidades que todos los módulos operativos necesitarán: organización, permisos, auditoría, documentos, jobs, outbox, idempotencia y códigos canónicos.

Flujo operativo esperado en Agro Ops

- Crear Organization, usuarios/roles/permisos y contexto de organización.
- Implementar audit_events y política de no-delete para operaciones confirmadas.
- Implementar jobs con estados, retries y next_attempt_at.
- Implementar transactional outbox.
- Implementar Documents + Storage metadata + SHA-256.
- Definir IDs internos UUID y códigos canónicos estables.

Rol de Finnegans y flujo entre sistemas

- Finnegans se modela como sistema externo con ExternalReference; todavía no se sincronizan documentos reales.
- Se define el formato estándar para registrar external_system, external_id, sync_version y last_sync_status.
- El futuro adapter utilizará outbox/jobs, nunca llamadas frágiles incrustadas dentro de la transacción de negocio.

Pruebas obligatorias de la etapa

- Outbox se crea en la misma transacción que el cambio de negocio.
- Retry duplicado no genera segundo efecto.
- Job fallido transiciona correctamente y puede reintentarse.
- Audit registra before/after y usuario.
- Document hash detecta duplicados o corrupción donde corresponda.
- Permisos bloquean operaciones no autorizadas en API aunque el frontend sea manipulado.

CRITERIO DE SALIDA

Existe un core reutilizable y probado; los módulos siguientes solo deben declarar sus reglas y casos de uso, no reinventar auditoría, jobs o integración.

10. Etapa 3 - Territorio, campañas y cartografía

Crear la base territorial del sistema porque agricultura, ganadería, maquinaria, costos y comercialización necesitan saber dónde ocurre cada evento. Se conserva el modelo v3.1: establecimiento -> lote base geográfico -> unidad operativa por campaña -> actividad/uso.

Flujo operativo esperado en Agro Ops

- Crear establecimiento y lotes base con geometría PostGIS.
- Importar o dibujar geometrías piloto; conservar identificador canónico estable.
- Crear campañas y unidades operativas (UOP) que puedan dividir/unir lotes sin modificar historia anterior.
- Asignar cultivo/uso a UOP por campaña.
- Mostrar mapa MapLibre; click en polígono abre ficha contextual.
- Agregar controles de superficie, estado y datos faltantes.

Rol de Finnegans y flujo entre sistemas

- Finnegans no es fuente de verdad territorial; Agro Ops mantiene geometrías y UOP.
- Si Finnegans necesita establecimiento/campaña/centro de costo como dimensión, se sincronizan códigos/referencias, no polígonos.
- Los códigos maestros que ambos mundos utilicen deben mapearse explícitamente y no por descripción libre.

Pruebas obligatorias de la etapa

- Geometría válida y SRID correcto.
- UOP contenida en el lote base y reglas de solapamiento.
- Suma de superficies no excede la base salvo excepción documentada.
- Crear campaña 1, subdividir; crear campaña 2 con división distinta; comprobar que campaña 1 permanece idéntica.
- E2E: mapa -> click -> ficha -> campaña/actividad correctas.
- Property tests sobre áreas/operaciones geométricas que puedan formalizarse.

CRITERIO DE SALIDA

Un establecimiento piloto puede navegarse por mapa y representar dos campañas con subdivisiones distintas sin pérdida de historia.

11. Etapa 4 - Stock operativo

Construir un ledger confiable para insumos y luego granos. El saldo nunca es la única fuente: se deriva de movimientos confirmados, transferencias, ajustes y reversión.

Flujo operativo esperado en Agro Ops

- Crear productos, unidades de medida y depósitos.
- Crear inventory_movements con signo, cantidad Decimal, origen, depósito y estado.
- Implementar ingreso, salida/consumo, transferencia, ajuste y reversión.
- Agregar saldo por producto/depósito y trazabilidad desde cada movimiento al documento origen.
- Bloquear mutaciones destructivas; una anulación crea la reversión necesaria.
- Preparar reservas/compromisos si luego son necesarias para granos.

Rol de Finnegan y flujo entre sistemas

- La compra/factura sigue naciendo en Finnegan; Agro Ops debe recibir el ingreso operativo por integración cuando el POC lo defina.
- Un consumo originado en Agro Ops debe poder generar solamente la consecuencia patrimonial que Finnegan requiera, sin recrear la labor ni ingresos internos ficticios.
- Mientras la integración no esté validada, el módulo se prueba con eventos simulados y conciliación manual controlada.

Pruebas obligatorias de la etapa

- Saldo = suma exacta de movimientos confirmados.
- Transferencia reduce un depósito y aumenta otro sin cambiar total global.
- Consumo y reversión restauran saldo original.
- Idempotencia de requests repetidos.
- Concurrencia: dos consumos simultáneos no violan reglas de saldo.
- No uso de float en cantidades.
- Mutations: stock $\geq$ requerido vs $>$ requerido debe ser detectado por tests.

CRITERIO DE SALIDA

Stock piloto reproducible desde cero, con saldos explicables por ledger y suite de regresión robusta.

12. Etapa 5 - Agricultura

Registrar qué se hizo, dónde, cuándo, con qué insumos/equipos y qué producción resultó, sin generar contabilidad ficticia por labores propias.

Flujo operativo esperado en Agro Ops

- Crear actividades agrícolas y catálogo de labores.
- Registrar labor por establecimiento + campaña + UOP, superficie y fecha.
- Asociar insumos: cada consumo confirma movimientos de stock en la misma operación lógica.
- Asociar equipo/contratista cuando corresponda.
- Registrar cosecha por UOP y generar ingreso de stock de grano.
- Mostrar labores, insumos y producción en la ficha del lote/mapa.

Rol de Finnegan y flujo entre sistemas

- Labores propias viven solamente en Agro Ops; no se envían como "Siembra propia" o "Ingreso por servicio propio" a Finnegan.
- Facturas de contratistas nacen en Finnegan y posteriormente se vinculan como costo real a la labor/actividad.
- Compras de insumos permanecen financieras en Finnegan; Agro Ops recibe cantidad/costo de referencia según integración definida.
- Cosecha nace en Agro Ops; la consecuencia patrimonial de grano en Finnegan se define en POC sin duplicar efectos.

Pruebas obligatorias de la etapa

- Crear labor sin UOP/campaña debe fallar.
- Superficie labor no puede exceder regla definida sin autorización explícita.
- Consumo de insumos y anulación revierten ledger de forma exacta.
- Cosecha genera exactamente un ingreso de stock por idempotency key.
- Editar datos permitidos mantiene trazabilidad; operaciones confirmadas no se eliminan.
- E2E: mapa -> UOP -> nueva labor -> insumos -> saldo stock -> ficha actualizada.

CRITERIO DE SALIDA

Un lote piloto puede recorrer labor -> consumo -> cosecha -> stock con trazabilidad completa y sin impactos contables ficticios.

13. Etapa 6 - Maquinaria y vehículos

Gestionar activos operativos, uso, horas/km, combustible y mantenimiento, distinguiendo costo real proveniente de Finnegan de tarifa/costo interno de gestión.

Flujo operativo esperado en Agro Ops

- Crear maestro de equipos y vehículos con unidad de uso (horas/km/ha).
- Registrar lecturas y eventos de uso vinculados a actividad/UOP/rodeo/logística.
- Registrar combustible operativo y relacionarlo con stock/factura según circuito.
- Crear mantenimiento preventivo y correctivo, paradas y reparaciones.
- Calcular costo real por hora/km y comparar contra tarifa interna.
- Mostrar historial y costos por equipo.

Rol de Finnegan y flujo entre sistemas

- Factura de combustible/repuesto/reparación permanece en Finnegan como costo real y deuda.
- Agro Ops recibe el costo y lo vincula a equipo/período/actividad; no genera duplicación financiera.
- Tarifas internas sirven para gestión y nunca se envían como ingresos propios a Contabilidad.
- Depreciación contable puede importarse como referencia de costo si se decide incluirla.

Pruebas obligatorias de la etapa

- Lecturas no pueden retroceder salvo corrección autorizada y auditada.
- Consumo combustible debe pertenecer a un equipo y período válido.
- Costo real recibido dos veces desde Finnegan no se duplica.
- Anulación/reasignación deja historial y recomputa indicadores.
- E2E: uso -> combustible/reparación -> costo -> asignación a actividad -> indicador.

CRITERIO DE SALIDA

Uno o dos equipos reales tienen uso, costos y asignación productiva conciliables contra Finnegan.

14. Etapa 7 - Ganadería

Representar existencias físicas y evolución productiva por establecimiento, categoría y rodeo/grupo, manteniendo compras/ventas fiscales en Finnegan.

Flujo operativo esperado en Agro Ops

- Crear categorías, rodeos/grupos y saldo inicial controlado.
- Implementar movimientos internos entre establecimientos/rodeos.
- Implementar cambios de categoría, pesadas, nacimientos/bajas si entran en alcance.
- Asociar alimentación e insumos al stock operativo.
- Registrar eventos económicos de compra/venta como referencias a documentos fiscales.
- Calcular existencias y, si aplica, kilos/producción por período.

Rol de Finnegan y flujo entre sistemas

- Compra/venta fiscal de hacienda nace en Finnegan; Agro Ops recibe o registra la consecuencia física/económica vinculada al documento real.
- Movimientos internos, pesadas y cambios de categoría no generan documentos financieros ficticios.

- Factura veterinaria/alimentación nace en Finnegans y el costo real se asigna en Agro Ops.
- Las existencias deben conciliarse periódicamente con la representación patrimonial que se defina.

Pruebas obligatorias de la etapa

- Ecuación de cabezas: inicial + altas/entradas - bajas/salidas +/- ajustes = final.
- Cambio de categoría conserva total de cabezas.
- Traslado conserva total global y cambia ubicación.
- Peso y cabezas no pueden quedar en estados imposibles definidos por el dominio.
- Compra/venta duplicada por retry no duplica stock.
- E2E: saldo inicial -> traslado -> cambio categoría/pesada -> venta -> existencia final.

CRITERIO DE SALIDA

Un establecimiento/categoría piloto puede conciliar existencias físicas y documentos fiscales sin doble carga manual.

15. Etapa 8 - Granos

Construir la representación física/comercial del grano desde la cosecha hasta depósitos, traslados y documentos posteriores, reutilizando el ledger de stock.

Flujo operativo esperado en Agro Ops

- Recibir stock desde cosecha y clasificar producto/campaña/ubicación.
- Implementar movimientos entre depósitos/ubicaciones y estados físicos/comerciales que se definan.
- Preparar entidades CPE, certificado y liquidación como relaciones externas/operativas, todavía sin conectar ARCA/Finnegans.
- Permitir trazabilidad desde kg de cosecha hasta movimientos comerciales.
- Agregar conciliación de kilos por origen, producto, campaña y depósito.

Rol de Finnegans y flujo entre sistemas

- Finnegans no dicta el ledger interno de grano.
- La representación patrimonial se concilia con Finnegans por cantidad y reglas acordadas.
- Las liquidaciones fiscales seguirán naciendo en Finnegans; esta etapa solamente prepara el modelo al que luego se importarán.
- ARCA se conectará después de que la entidad interna CPE esté estable.

Pruebas obligatorias de la etapa

- Cosecha genera el mismo stock esperado por campaña/UOP.
- Traslado conserva total global de kg.
- Reversión de traslado restaura ubicación anterior.
- No se permiten referencias a campañas/productos inconsistentes.
- Property-based tests para conservación de kilos a través de secuencias de movimientos.

CRITERIO DE SALIDA

El stock de grano puede explicarse desde cosecha y movimientos internos sin depender todavía de integraciones externas.

16. Etapa 9 - Comercialización de granos

Modelar contratos, CPE, certificados, liquidaciones y fletes como un grafo flexible que soporte el orden real de los documentos y evite las restricciones detectadas en el circuito actual.

Flujo operativo esperado en Agro Ops

- Crear contratos con kilos comprometidos, producto, campaña, contraparte y condiciones relevantes.
- Permitir vincular CPE anteriores o posteriores al contrato y vínculos parciales.

- Permitir múltiples CPE por contrato y múltiples certificados/liquidaciones según caso.
- Modelar liquidación parcial/final como relaciones que no obliguen un orden rígido.
- Agregar fletes operativos y costos asignables.
- Mostrar balance: comprometido, enviado, certificado, liquidado, pendiente.

Rol de Finnegan y flujo entre sistemas

- Contrato operativo puede ser dueño Agro Ops, pero cualquier documento fiscal/contable que Finnegan necesite se sincroniza por adapter.
- Liquidación oficial permanece en Finnegan y se replica a Agro Ops con kilos, precio, gastos y referencias; no se emite dos veces.
- Certificados deben conservar relación con CPE y stock según el circuito validado.
- Flete operativo puede nacer en Agro Ops; factura del transportista permanece en Finnegan y su costo real vuelve a Agro Ops.

Pruebas obligatorias de la etapa

- Casos de orden: CP->contrato, contrato->CP, CP parcial, contrato parcial, liquidación parcial antes/después de otros documentos.
- Una CPE puede vincularse exactamente a los kg permitidos sin sobreasignación.
- Reintento de vinculación/importación no duplica relación.
- Liquidación parcial/final actualiza el balance esperado sin borrar documentos previos.
- Caso de regresión del problema actual: liquidación final debe poder reutilizar CPE ya vinculadas y no exigir selección redundante si el modelo ya las conoce.

CRITERIO DE SALIDA

El modelo interno representa todos los casos comerciales conocidos sin depender del orden de carga de Finnegan.

17. Etapa 10 - Integración ARCA / Carta de Porte Electrónica

Conectar Agro Ops con ARCA para importar y actualizar CPE de forma controlada. V1 será read-only: consultar, normalizar, almacenar referencias/documento y relacionar con la operación; no emitir CPE desde Agro Ops.

Flujo operativo esperado en Agro Ops

- Implementar adapter ARCA aislado: autenticación WSAA, cliente WSCPE, parser y mapeo a dominio.
- Guardar external_id/CTG, estado, pesos, producto, fechas, destino/transportista y demás campos necesarios del modelo.
- Guardar PDF/adjunto cuando el servicio lo permita y registrar hash.
- Crear bandeja de CPE nuevas/no vinculadas para que el usuario asocie establecimiento/campaña/contrato cuando no pueda inferirse de forma segura.
- Crear cron de descubrimiento/sincronización y worker para procesar jobs/retries.
- Actualizar cambios de estado sin duplicar CPE.

Rol de Finnegan y flujo entre sistemas

- ARCA es la fuente legal; Agro Ops guarda una réplica operativa y relaciones internas.
- Finnegan no debe convertirse en intermediario obligatorio de la importación ARCA; ambos sistemas pueden conservar las referencias que necesiten.
- La relación CPE -> contrato/campaña/UOP se administra en Agro Ops y luego puede enviarse a Finnegan si es necesaria para su circuito.

Pruebas obligatorias de la etapa

- Fixtures SOAP/XML anonimizadas en cada push.
- Mock WSAA/WSCPE: éxito, timeout, SOAP fault, TA expirado, respuesta incompleta, CPE modificada.
- Misma CPE importada N veces produce una sola entidad y actualizaciones versionadas.

- Worker reiniciado a mitad de proceso retoma sin duplicar.
- Nightly/manual contra homologación ARCA.
- E2E staging: sync -> bandeja -> vincular -> ficha contrato/CPE.

CRITERIO DE SALIDA

POC ARCA aprobado en homologación y sincronización de CPE real controlada sin duplicados ni dependencia de operaciones manuales repetitivas.

18. Etapa 11 - Integración Finnegans

Integrar solamente los eventos que realmente deben cruzar entre la operación y la contabilidad. El adapter debe ser reemplazable y no contaminar el dominio con detalles internos de GO.

Flujo operativo esperado en Agro Ops

- Definir contrato de integración por evento: qué entra, qué sale, owner, campos, idempotency key y reversión.
- Implementar capacidades de lectura de costos/documentos y escritura de consecuencias operativas según mecanismos disponibles.
- Usar outbox para eventos Agro Ops -> Finnegans y jobs para Finnegans -> Agro Ops.
- Registrar ExternalReference con IDs, versión y estado de sync.
- Crear monitor de pendientes/errores/mapeos y acción de retry segura.

Rol de Finnegans y flujo entre sistemas

- Finnegans conserva facturas, impuestos, bancos, proveedores/clientes, liquidaciones fiscales y contabilidad.
- Agro Ops recibe costos reales, compras/ventas fiscales y liquidaciones necesarias para gestión.
- Agro Ops envía solo consecuencias definidas de stock/producción cuando corresponda; labores propias y tarifas internas no generan asientos ficticios.
- La solución debe usar API/mecanismos estándar/import-export soportados; no depende de BProc.

Pruebas obligatorias de la etapa

- Contract tests con payloads reales del mecanismo elegido.
- Alta, modificación, anulación y retry idempotente.
- Error de mapeo debe bloquear/avisar sin crear datos incorrectos.
- Documento recibido dos veces no duplica costo/stock.
- Caída temporal de Finnegans no pierde outbox.
- POC específico de consumo de insumos, liquidación, flete, maquinaria y ganadería.

CRITERIO DE SALIDA

Los POC críticos demuestran carga manual única y conciliación estable entre Agro Ops y Finnegans.

19. Etapa 12 - Costos, resultados y Margen Bruto

Construir resultados solamente cuando las fuentes de cantidades y costos sean confiables. Se distingue costo real recibido de Finnegans, costo interno/tarifa de gestión y asignación productiva.

Flujo operativo esperado en Agro Ops

- Crear motor de asignación de costos por establecimiento, campaña, UOP, actividad, rodeo, equipo o logística.
- Importar costos reales de contratistas, fletes, reparaciones, veterinaria, etc.
- Aplicar tarifas internas de labores/maquinaria donde corresponda sin tratarlas como transacciones contables.
- Calcular Margen Bruto agrícola y resultados ganaderos/maquinaria según reglas aprobadas.
- Permitir drill-down desde resultado hasta eventos, movimientos y documentos origen.

Rol de Finnegans y flujo entre sistemas

- Finnegans es fuente del costo financiero real cuando existe documento real.
- Agro Ops puede tener valorizaciones internas distintas para gestión, pero nunca enviarlas como ingresos/egresos ficticios a Contabilidad.
- La conciliación debe poder explicar diferencias de criterio entre resultado productivo y resultado contable.

Pruebas obligatorias de la etapa

- Reconciliar suma de componentes contra total del resultado.
- Mismo costo externo importado dos veces cuenta una sola vez.
- Anulación/corrección de documento actualiza el resultado sin borrar historia.
- Cambiar tarifa interna afecta gestión pero no genera outbox contable.
- Tests con datasets cerrados donde el MB esperado se calcula manualmente y debe coincidir al centavo/unidad definida.

CRITERIO DE SALIDA

Un establecimiento/campaña piloto produce un MB explicable y reproducible con drill-down completo a costos y cantidades.

20. Etapa 13 - Reportes, conciliación y alertas

Convertir las reglas de consistencia en controles visibles y recurrentes. La confianza del sistema depende de detectar diferencias, no de asumir que la sincronización siempre funciona.

Flujo operativo esperado en Agro Ops

- Crear dashboard de conciliación por dominio: stock insumos, stock granos, cabezas/kilos, costos, documentos y sync.
- Crear alertas por jobs DEAD, mapeos faltantes, CPE sin vincular, superficies inconsistentes, lecturas anómalas y diferencias de stock.
- Crear reportes de cierre por período y exportaciones controladas.
- Agregar historial de conciliaciones y responsable de resolución.

Rol de Finnegans y flujo entre sistemas

- Comparar Agro Ops con Finnegans para cantidades/costos que deban coincidir.
- Comparar CPE de Agro Ops con ARCA para estado/documento legal.
- No exigir que valorizaciones internas coincidan con contabilidad cuando la política dice que son distintas; sí exigir que la diferencia sea explicable.

Pruebas obligatorias de la etapa

- Dataset con diferencias conocidas debe producir exactamente las alertas esperadas.
- Cerrar una alerta requiere causa/resolución auditada.
- Cron de conciliación puede ejecutarse repetidamente sin duplicar alertas activas.
- Pruebas de tolerancias y redondeos.

CRITERIO DE SALIDA

Gerencia puede identificar pendientes y diferencias desde un único panel, con trazabilidad a la causa y acción correctiva.

21. Etapa 14 - Piloto, migración y cutover

Adoptar Agro Ops sin poner en riesgo la operación vigente. El nuevo sistema se valida en paralelo hasta cumplir un Readiness Gate; recién entonces se convierte en fuente operativa oficial.

Flujo operativo esperado en Agro Ops

- Elaborar inventario de maestros/saldos iniciales e histórico que se migrará.
- Elegir piloto: un establecimiento/campaña, un grupo ganadero y uno/dos equipos, más un circuito de granos representativo.

- Operar en paralelo y comparar resultados contra el procedimiento vigente.
- Capacitar usuarios y publicar guía operativa por evento.
- Definir fecha de corte, congelamiento de saldos, migración final y rollback.
- Pasado el cutover, eliminar doble carga manual y monitorizar diferencias intensivamente.

Rol de Finnegans y flujo entre sistemas

- Finnegans continúa siendo el sistema contable/fiscal antes, durante y después del cutover.
- La operación actual no se desarma hasta que los POC, pruebas y conciliaciones estén aprobados.
- El rollback debe poder detener sync y volver temporalmente al circuito anterior sin pérdida de documentos contables.

Pruebas obligatorias de la etapa

- Rehearsal de migración completo en staging.
- Comparación de saldos/campañas/documentos antes y después.
- Pruebas de rollback y reintento de sync.
- E2E de los flujos críticos con datos representativos.
- Performance básica con volumen superior al esperado para 10 usuarios.
- Security review y verificación de backups/restore.

CRITERIO DE SALIDA

Readiness Gate aprobado: maestros, saldos, seis+ POC funcionales, ARCA/Finnegans estables, usuarios capacitados, rollback probado y CI/monitorización en verde.

22. Flujos operativos end-to-end

22.1 Compra de insumo -> stock -> consumo agrícola

1. FINNEGANS: factura/recepción de compra (fuente financiera)
2. Integración -> Agro Ops: producto + cantidad + depósito + costo/referencia
3. AGRO OPS: movimiento +5.000 L en inventory ledger
4. Usuario registra labor en UOP de campaña
5. AGRO OPS: movimiento -280 L ligado a labor
6. Si corresponde, outbox -> Finnegans: consecuencia patrimonial del consumo
7. Conciliación: cantidad por producto/depósito + trazabilidad de documento

REGLA

La labor propia y su tarifa interna no se envían a Finnegans. Solo cruza la consecuencia real que se haya definido en el POC.

22.2 Labor contratada y costo real

```

AGRO OPS: se registra la labor operativa aunque la factura todavía no exista
      |
      v
FINNEGANS: llega factura del contratista -> costo/deuda real
      |
      v
      | integración por referencia/código
      v
AGRO OPS: costo real se vincula a labor/campaña/UOP
      |
      v
Margen Bruto / análisis de costo
  
```

22.3 Cosecha -> stock de grano

```

Usuario confirma cosecha en UOP
-> harvest confirmada
-> +kg en grain inventory ledger
-> audit event
-> outbox para consecuencia Finnegans si POC lo requiere
-> ficha de lote muestra producción y stock trazable

```

22.4 Contrato -> CPE -> certificado -> liquidación

```

AGRO OPS: contrato operativo (puede existir antes o después de la CPE)
|
ARCA: CPE legal -> importación read-only -> Agro Ops CPE
|
AGRO OPS: vínculo CPE <-> contrato/campaña/lote; puede ser parcial o múltiple
|
Certificado: relación física/comercial según circuito validado
|
FINNEGANS: liquidación fiscal oficial
|
| integraci3n
v
AGRO OPS: kilos/precio/gastos + relación con contrato/CPE/certificados
|
Resultado / stock / conciliaci3n

```

22.5 Reparaci3n de maquinaria

```

AGRO OPS: orden/registro operativo de mantenimiento
FINNEGANS: factura proveedor -> costo real
Integraci3n -> Agro Ops: costo vinculado a equipo/período
Agro Ops -> asignaci3n a actividad/establecimiento
Indicadores: costo/hora, disponibilidad, mantenimiento acumulado

```

22.6 Venta de hacienda

```

FINNEGANS: documento fiscal de venta
|
| integraci3n / referencia
v
AGRO OPS: baja física de categorí3a/rodeo + importe/costo relevante
|
Existencia final / resultado ganadero / conciliaci3n

```

23. Matriz de pruebas y precisi3n esperada

Dominio	Invariantes críticas	Tipo de test prioritario
Territorio	Historia de campañas, área, contenci3n, no solapamiento indebido.	PostGIS integration + E2E mapa + property tests selectivos.
Stock	Saldo por ledger, conservaci3n en transferencias, reversi3n exacta, concurrencia.	Unit + property + DB transactions + mutation.
Agricultura	Labor en contexto v3lido, consumo ligado, cosecha única/idempotente.	Application/API + DB + E2E.
Maquinaria	Lecturas coherentes, costo único, asignaci3n trazable.	Unit + DB + integration Finnegans.
Ganadería	Conservaci3n de cabezas, traslados, cambios categorí3a, stock no duplicado.	Property + DB + E2E.
Granos	Conservaci3n de kg, vínculos consistentes, reversi3n.	Property + DB + mutation.
Comercial	Orden flexible de documentos, asignaci3n	Scenario tests + API + E2E.

	parcial, no sobreasignación.	
ARCA	Parser exacto, idempotencia, retry, estados y errores SOAP.	Fixtures + mock server + homologación nightly.
Finnegans	Alta/modificación/anulación/retry, mapeos y no duplicación.	Contract tests + staging POC.
Costos/MB	Exactitud Decimal, componentes explicables, correcciones.	Golden datasets + mutation + regression.
Seguridad	Permiso real en backend; secrets no expuestos.	API auth tests + dependency/security scans.
Migrations	DB nueva y upgrade de versión anterior.	CI con PostgreSQL/PostGIS real.

23.1 Golden datasets

Para reglas económicas y de conciliación se crearán datasets pequeños, cerrados y revisados manualmente. Cada dataset incluirá inputs, eventos, saldos esperados y explicación. Si una refactorización cambia el resultado sin que exista una decisión de negocio aprobada, el test debe fallar.

23.2 Casos de borde obligatorios

- Cero, límites exactos y redondeos Decimal.
- Request repetida con misma idempotency key.
- Operación concurrente sobre el mismo stock/recurso.
- Anulación después de sincronización parcial.
- Sistema externo caído durante un commit ya confirmado.
- Mapeo maestro faltante.
- Importación de documento externo ya existente pero modificado.
- Migration con datos previos reales.
- Usuario sin permisos intentando llamar API directamente.
- Entidad histórica referenciada por una campaña cerrada.

24. ADR, cambios de arquitectura y evolución futura

La arquitectura puede evolucionar, pero no debe hacerlo de forma accidental. Todo cambio significativo se registra como Architecture Decision Record (ADR) antes de implementarse.

ADR inicial	Decisión
ADR-001	Rust + Axum para backend.
ADR-002	Monolito modular; no microservicios en V1.
ADR-003	PostgreSQL/PostGIS como base operativa.
ADR-004	Docker como entorno reproducible y artifact de deployment.
ADR-005	Supabase para DB/Auth/Storage en staging/production.
ADR-006	Vercel frontend; Railway API/worker/jobs.
ADR-007	Inventory/Grain ledger de movimientos.
ADR-008	Transactional outbox + jobs PostgreSQL; sin Redis inicialmente.
ADR-009	ARCA CPE V1 read-only.
ADR-010	Finnegans detrás de adapter; sin dependencia de BProc.
ADR-011	PWA online-first; offline complejo fuera de V1.
ADR-012	GitHub + required CI checks como gate de calidad.

24.1 Fuera de V1, pero compatible con el diseño

- Offline-first completo y resolución de conflictos.

- App móvil nativa.
- Emisión/modificación de CPE desde Agro Ops si se justifica.
- Automatización avanzada con sensores/telemetría.
- ML/forecasting en servicio Python separado si aporta valor.
- Extracción de un módulo a microservicio solo si escala/equipos independientes lo justifican.
- Multi-organización real si el negocio lo necesita.

25. Decisiones pendientes y POC

Tema	Qué se debe validar antes de producción
Finnegans - stock insumos	Documento/mecanismo estándar que reciba la consecuencia de consumo sin contabilidad ficticia de labor.
Finnegans - cosecha/grano	Cómo representar la consecuencia patrimonial y mantener cantidades conciliadas.
Finnegans - liquidación	Cómo importar a Agro Ops kilos/precio/gastos/refs de la liquidación fiscal sin duplicar efectos.
Finnegans - certificados	Empresa/sistema dueño del certificado y relación con CPE/stock en el circuito definitivo.
Finnegans - integración técnica	API/import-export/mecanismo estándar disponible en GO; autenticación, límites, modificación/anulación.
ARCA - descubrimiento CPE	Método final para detectar nuevas CPE propias/destino y frecuencia del cron.
ARCA - certificados	Gestión segura de certificado, vencimiento y renovación.
Territorio	Fuente inicial de geometrías y regla para cambios físicos reales del lote base.
Ganadería	Nivel de detalle final: individual, categoría, rodeo o combinación.
Stock negativo	Política por producto/depósito y excepciones autorizadas.
Costos	Reglas exactas de tarifas internas, depreciación y criterios de asignación.
Migración	Profundidad histórica, saldos iniciales y fecha de cutover.

26. Roadmap resumido y orden de ejecución

Orden	Etapas	Resultado visible/probable
0	Repositorio y gobierno	GitHub protegido, ADR, estructura y primer CI.
1	Walking Skeleton	Web + Rust API + worker + PostGIS funcionando local/staging.
1.5	Diseño de datos / DER	DER maestro conceptual versionado; entidades, cardinalidades, invariantes y mapa de implementación por etapa.
2	Core transversal	Auth, roles, audit, jobs, outbox, documents.
3	Territorio	Mapa real de establecimiento/lotas/campañas.
4	Stock	Ledger de insumos confiable.
5	Agricultura	Labor -> insumos -> cosecha.
6	Maquinaria	Uso, combustible, mantenimiento y costos.

Orden	Etapas	Resultado visible/probable
7	Ganadería	Existencias y movimientos.
8	Granos	Stock físico y movimientos.
9	Comercialización	Contratos/CPE/certificados/liquidaciones como grafo operativo.
10	ARCA	Importación automática CPE read-only.
11	Finnegans	Carga única + sync de consecuencias/costos/documentos.
12	Costos/MB	Resultados productivos completos.
13	Conciliación	Dashboard de diferencias/alertas.
14	Piloto/cutover	Operación paralela, readiness gate y adopción.

PRIMER BLOQUE DE TRABAJO RECOMENDADO

No comenzar por Agricultura. Ejecutar primero Etapas 0, 1, **1.5** y 2 hasta tener GitHub + Docker + CI + Rust API + worker + PostGIS + staging + **DER maestro aprobado** + audit/outbox. Después construir Territorio como primera vertical operativa y probar el sistema real a medida que se agregan features.

26.1 Criterio para acomodar el plan

El diseño es deliberadamente adaptable. Se puede cambiar una regla funcional o el orden fino de subfeatures si los usuarios descubren una forma mejor de operar. Lo que no debe cambiar sin decisión explícita es la separación de responsabilidades, la trazabilidad, la idempotencia, el ledger, la auditoría y la disciplina de tests/CI. Si una modificación exige cambiar arquitectura, se documenta con ADR y se prueba antes de expandirla al resto del sistema.

26.2 Resultado objetivo final

AGRO OPS OPERATIVO

La empresa dispone de un sistema propio para operar territorio, agricultura, stock, maquinaria, ganadería, granos y comercialización; ARCA aporta la CPE legal; Finnegans conserva contabilidad/fiscalidad; y cada integración es observable, idempotente, conciliable y cubierta por pruebas automáticas. Cada push a GitHub vuelve a demostrar que lo nuevo no rompe lo anterior.

Fin del documento.

ACTUALIZACIÓN v4.2 - REQUISITO HTTP/2 MÍNIMO

CARÁCTER DE LA ACTUALIZACIÓN

La versión 4.2 conserva íntegramente el Plan Integral v4.1 y agrega un requisito técnico de transporte HTTP. No elimina, sustituye, posterga ni reduce la prioridad de ninguna decisión, etapa, prueba, integración, ADR o criterio de salida existente.

A. Nuevo principio no negociable

Principio	Regla
HTTP/2 mínimo en tráfico público	Todo endpoint HTTP público de Agro Ops en staging y producción deberá negociar HTTP/2 o una versión superior cuando la infraestructura utilizada lo permita. HTTP/1.1 puede seguir utilizándose en desarrollo local, health checks internos o enlaces internos cuando resulte conveniente o sea impuesto por la plataforma. Si un endpoint público no puede ofrecer HTTP/2+, la limitación deberá quedar documentada explícitamente y, si afecta arquitectura, mediante ADR.

B. Arquitectura y deployment

- El requisito aplica a la interfaz pública cliente -> plataforma. Vercel/Railway o el proxy/edge equivalente pueden terminar TLS y negociar HTTP/2+ antes de reenviar la solicitud a Axum.
- No se exige que el proceso Axum exponga HTTP/2 directamente si el edge ya satisface el requisito. Se preserva la simplicidad del backend y la arquitectura existente.
- HTTPS continúa siendo obligatorio para tráfico público. La selección efectiva del protocolo se realiza por negociación de la plataforma y del cliente.
- HTTP/3 puede habilitarse si el proveedor lo ofrece de forma estable, pero no se convierte en requisito de V1 ni desplaza trabajo funcional, de datos, seguridad, testing o integración.

C. Validación obligatoria en staging/producción

- Smoke test del endpoint público que verifique HTTPS y negociación HTTP/2 o superior.
- La verificación se incorpora a la checklist de deployment de la Etapa 1 y a regresiones de infraestructura cuando cambie Vercel, Railway, proxy, dominio o TLS.
- Una degradación pública a HTTP/1.1 no debe aceptarse silenciosamente: se corrige o se documenta la restricción de plataforma y su impacto.

REGLA DE PRIORIDAD

Este requisito es aditivo. No modifica el orden del roadmap ni rebaja requisitos previos. En particular, no desplaza PostgreSQL/PostGIS, SQLx, readiness real, migrations, CI, pruebas, observabilidad, seguridad, staging, DER, audit, outbox, idempotencia ni integraciones. Se implementa en el momento en que exista un endpoint público de staging/producción que deba exponerse por HTTP.

E. Secciones del Plan v4.1 que quedan ampliadas

- 2. Principios no negociables: se agrega "HTTP/2 mínimo en tráfico público".
- 4. Arquitectura de desarrollo, entornos y deployment: staging/production deben verificar negociación HTTP/2+ en el edge.
- 5. GitHub, CI/CD y sistema de pruebas: smoke/deployment checks incorporan verificación del protocolo público.
- 8. Etapa 1 - Walking Skeleton: el criterio de staging incluye endpoint HTTPS público negociando HTTP/2 o superior.
- 24. ADR y evolución futura: solo una excepción estructural o cambio de estrategia de transporte que altere arquitectura requiere ADR.

ACTUALIZACIÓN v4.3 - INTERNAL / TECHNICAL CONSOLE

CARÁCTER DE LA ACTUALIZACIÓN

La versión 4.3 conserva íntegramente el Plan Integral v4.2 y agrega una capacidad transversal de observabilidad y diagnóstico dentro de Agro Ops: una **Internal / Technical Console**. No elimina, sustituye, posterga ni reduce la prioridad de ninguna decisión, etapa, prueba, integración, ADR o criterio de salida existente. Tampoco convierte la consola en una nueva fuente de verdad ni modifica el orden del roadmap.

A. Nueva capacidad transversal: Internal / Technical Console

Agro Ops dispondrá de un área técnica interna para visualizar el estado real de la plataforma y, a medida que se implementen las etapas posteriores, centralizar diagnósticos de infraestructura, jobs, outbox e integraciones. Su objetivo es reducir la dependencia de inspecciones manuales en Docker, PostgreSQL, Railway, Supabase o logs para comprobaciones rutinarias, sin ocultar ni reemplazar esas herramientas.

Principio	Regla
Consola técnica interna	Las capacidades técnicas visibles desde la aplicación se agrupan bajo /internal . La consola muestra estado derivado de fuentes reales; no simula estados saludables ni mantiene una copia paralela de datos operativos.
Read-only por defecto	Las vistas de diagnóstico son de solo lectura salvo acciones de remediación explícitamente diseñadas. Cualquier retry, requeue o acción equivalente deberá ser autorizada, idempotente y auditada.
Seguridad y mínimo privilegio	En staging/producción el acceso será restringido a roles técnicos/administrativos. La autorización efectiva se aplica en backend cuando corresponda; la UI no es una barrera de seguridad.
No exposición de secretos	Nunca se muestran private keys, certificados completos, tokens, passwords, connection strings, secretos de infraestructura ni payloads sensibles sin redacción.

B. Estructura prevista

- **/internal** - entrada de la consola técnica y navegación entre capacidades disponibles.
- **/internal/system-status** - estado de Web, API, Database, Worker, versión/build y entorno.
- **/internal/jobs** - Etapa 2: jobs pendientes, running, failed/dead, retries y timestamps relevantes.
- **/internal/outbox** - Etapa 2: eventos pendientes/fallidos y trazabilidad de entrega.
- **/internal/integrations/arca** - Etapa 10: conectividad, última sincronización, errores y estado del adapter, sin exponer credenciales.
- **/internal/integrations/finnegans** - Etapa 11: conectividad, mapeos, pendientes, errores y retry seguro.
- **/internal/audit** - cuando se materialice el audit transversal, vistas técnicas de trazabilidad apropiadas para soporte y diagnóstico.

La estructura es incremental: no se crean pantallas, tablas ni abstracciones "por adelantado". Cada módulo aparece cuando la etapa que lo necesita materializa su fuente de datos y sus reglas.

C. Alcance inicial en Etapa 1 - Walking Skeleton

- Crear la sección **/internal** y la página **/internal/system-status**.
- Mostrar Web, API, Database y Worker con estados obtenidos de verificaciones reales.
- Mostrar versión del backend y, cuando esté disponible sin complejidad innecesaria, environment/build o commit de deployment.
- El worker persiste su heartbeat en PostgreSQL; la API clasifica healthy, stale o unavailable según antigüedad real.
- Una falla de red o dependencia se representa como unavailable/degraded y no debe hacer caer la página completa.
- En local la consola puede permanecer accesible durante el Walking Skeleton. La protección por rol se incorpora con la capacidad de Auth/roles definida por el plan; staging/production no deberán exponer información sensible sin control de acceso.

D. Relación con System Status y observabilidad

System Status deja de ser una página aislada y pasa a ser la primera herramienta de la consola. Logging/tracing, **request_id**, **correlation_id** y OpenAPI conservan su alcance original. La consola puede enlazar o resumir estas capacidades, pero no sustituye logs estructurados, métricas de plataforma ni herramientas de infraestructura cuando se necesita diagnóstico profundo.

ACTUALIZACIÓN v4.3 - IMPLEMENTACIÓN Y PRUEBAS

E. Implementación incremental por etapa

Etapa	Ampliación de la consola
1 - Walking Skeleton	Internal home + System Status real: Web, API, DB, Worker, version/build/environment cuando corresponda.
2 - Núcleo transversal	Jobs, outbox y diagnóstico de audit/idempotencia. Las acciones de retry/requeue solo se agregan con permisos, idempotencia y audit.
10 - ARCA	Estado del adapter, última sync, pendientes/errores y health operativo. Nunca mostrar certificados privados, tokens ni secretos.
11 - Finnegans	Estado del adapter, mapeos faltantes, pendientes/errores y retry seguro, reutilizando ExternalReference, jobs y outbox.
13 - Conciliación	La consola técnica puede enlazar al dashboard de conciliación, pero no duplica el producto funcional de alertas/diferencias previsto para gerencia/operación.

F. Pruebas y validaciones obligatorias

- System Status usa datos reales y distingue correctamente healthy, stale, degraded/unavailable y ausencia de datos.
- Heartbeat del worker se persiste/actualiza de forma determinística; detener el worker lo vuelve stale y reiniciarlo lo devuelve a healthy sin registros duplicados.
- Fallas de API, DB o red no rompen el render completo de la consola y nunca se presentan como OK.
- La consola no expone secretos ni configuración sensible en respuestas de API, HTML, logs del navegador o errores visibles.
- Cuando se incorpore autorización, un usuario sin permiso no accede a endpoints internos aunque manipule directamente la URL o el frontend.
- Acciones técnicas futuras (retry/requeue) tienen tests de autorización, idempotencia, audit y comportamiento ante repetición.
- En staging/production, los smoke tests públicos mantienen el requisito HTTP/2+ agregado por v4.2; la consola no modifica ese requisito.

G. Criterio de diseño de la consola

- No introducir una plataforma de observabilidad propia compleja en V1; reutilizar PostgreSQL, endpoints internos y metadata que el sistema ya necesita.
- No consultar Docker Engine ni depender de nombres de containers para determinar salud: el mecanismo debe funcionar igual en local y en Railway/Supabase.
- No almacenar “estado calculado” si puede derivarse con seguridad de heartbeats, jobs, outbox y timestamps existentes.
- Evitar que la consola se convierta en un segundo panel operativo para usuarios finales; las pantallas de negocio continúan en sus módulos correspondientes.

REGLA DE PRIORIDAD

La Internal / Technical Console es una capacidad transversal y aditiva. No crea una nueva etapa ni cambia el roadmap. Se implementa progresivamente usando las fuentes reales que cada etapa ya introduce. No desplaza Auth, PostgreSQL/PostGIS, SQLx, migrations, CI, testing, staging, DER, audit, outbox, idempotencia, seguridad, ARCA, Finnegans ni las verticales operativas.

H. Secciones del Plan v4.2 que quedan ampliadas

- **2. Principios no negociables:** se agrega consola técnica interna, read-only por defecto, mínimo privilegio y no exposición de secretos.
- **3. Arquitectura objetivo:** Web/PWA incorpora una superficie interna de diagnóstico sin cambiar responsabilidades del dominio o adapters.
- **5. GitHub, CI/CD y sistema de pruebas:** se agregan tests de estados reales, fallas parciales, seguridad y acciones técnicas futuras.
- **8. Etapa 1 - Walking Skeleton:** System Status se implementa bajo `/internal/system-status` y utiliza heartbeat persistido del worker.
- **9. Etapa 2 - Núcleo transversal:** jobs/outbox/audit podrán exponerse de forma técnica y controlada en la consola.
- **17/18. Integraciones ARCA/Finnegans:** sus monitores técnicos se integran bajo la consola sin exponer secretos.
- **20. Reportes, conciliación y alertas:** se conserva como capacidad funcional separada; la consola técnica puede enlazarla pero no duplicarla.
- **24. ADR:** esta ampliación no requiere un ADR nuevo porque no cambia la arquitectura acordada; una modificación estructural futura sí deberá documentarse.

I. Roadmap

El orden de ejecución continúa sin cambios: **0 -> 1 -> 1.5 -> 2 -> 3 ... -> 14**. Durante Etapa 1 se materializa únicamente la primera pantalla de la consola; el resto aparece en la etapa que crea sus datos y casos de uso.

ACTUALIZACION v4.4 - APLICACION PRIVADA, AUTENTICACION Y AUTORIZACION POR CAPACIDADES

CARACTER DE LA ACTUALIZACION

La version 4.4 conserva integramente el Plan Integral v4.3 y agrega reglas explicitas de acceso privado, identidad individual, autenticacion real y autorizacion por capacidades. No elimina, sustituye, posterga ni reduce la prioridad de ningun requerimiento, etapa, prueba, integracion, ADR o criterio de salida existente. Esta actualizacion precisa como se protege una aplicacion de uso interno de una unica empresa durante V1 y como se atribuyen las acciones a cada usuario.

A. Modelo de acceso y propiedad de los datos

Principio	Regla
Aplicacion privada por defecto	Las pantallas y APIs operativas de Agro Ops requieren una identidad autenticada, salvo endpoints tecnicos publicos expresamente declarados. No existe acceso anonimo a dashboards, modulos operativos ni consola interna.
Datos compartidos de la empresa	En V1 los datos operativos pertenecen a la empresa/organizacion, no al usuario que creo el registro ni al usuario que genero o compartio un link. Un usuario autenticado puede acceder a un recurso si su permiso de seccion/accion lo autoriza.
Identidad individual obligatoria	Cada persona utiliza su propia cuenta. No se comparten credenciales. La identidad autenticada se propaga al backend para atribuir creaciones, modificaciones, anulaciones y acciones tecnicas.
Autorizacion por capacidades	Los permisos se expresan por seccion y accion, por ejemplo view, create, edit, adjust, manage o acciones especificas del dominio. Los roles son agrupaciones de capacidades y no reemplazan la verificacion efectiva del permiso.
Backend como barrera de seguridad	Ocultar una ruta, boton o menu en Next.js mejora UX pero no constituye autorizacion. Rust valida identidad y permisos en los endpoints privados; una llamada directa a la API no puede eludir el control.
Sin auto-registro publico	No se ofrece sign-up abierto. Las cuentas son creadas, invitadas o habilitadas por administradores autorizados mediante el mecanismo definido para Supabase Auth.

REGLA DE LINKS Y NAVEGACION

Un link privado no pertenece a quien lo creo. Si un usuario sin sesion abre /dashboard, /inventory o cualquier deep link privado, se redirige a login y, tras autenticarse, puede volver a la URL original. Si ya esta autenticado, el acceso depende de sus capacidades, no del usuario que compartio el link.

B. Alcance minimo de autenticacion en Etapa 1 - Walking Skeleton

- **Supabase Auth real** en staging y production, usando proyectos/credenciales separadas por ambiente.
- Login con una cuenta habilitada, logout y sesion persistente con manejo correcto de expiracion/renovacion segun el SDK elegido.
- Las rutas privadas de Next.js detectan ausencia de sesion y redirigen a **/login**; se conserva de forma segura la ruta de retorno para continuar luego del login.
- El frontend envia el access token de la sesion en las llamadas a endpoints privados del backend.
- Rust valida el token/identidad antes de atender endpoints privados. Una sesion inexistente, vencida o invalida no obtiene datos internos.
- No se expone en el navegador ninguna service-role key, private key, password, connection string ni secreto de infraestructura.
- La pagina **/internal** y **/internal/system-status** requieren autenticacion desde Etapa 1; los permisos tecnicos finos se agregan en Etapa 2.
- Los endpoints publicos se limitan a los que infraestructura realmente necesite, por ejemplo **/health**, **/ready** y/o **/version**, manteniendo respuestas minimas y no sensibles. **/internal/worker/status** y futuros endpoints internos no son publicos por defecto.
- No se implementan todavia roles finos ni ownership por usuario: Etapa 1 demuestra identidad, sesion y frontera privada real.

ACTUALIZACION v4.4 - AUTORIZACION Y AUDITORIA

C. Etapa 2 - permisos por seccion y accion

La Etapa 2 mantiene Organization, usuarios, roles y permisos del Plan original, pero se precisa que el modelo inicial es una unica empresa/organizacion operativa con datos compartidos. Organization sigue existiendo como contexto y limite de datos; no se introduce un silo de datos por usuario.

Capacidad	Semantica V1
dashboard:view	Ver el dashboard comun de la empresa.
agriculture:view / create / edit	Consultar, cargar o modificar informacion agricola segun la accion autorizada.
inventory:view / adjust	Consultar stock y ejecutar ajustes/acciones sensibles solo cuando exista permiso especifico.
livestock:view / edit	Consultar o modificar ganaderia segun capacidad.
machinery:view / edit	Consultar o modificar maquinaria y vehiculos segun capacidad.
commercial:view / edit	Consultar o modificar contratos, CPE, certificados, liquidaciones y fletes segun etapa y permiso.
configuration:view / manage	Separar consulta de configuracion de cambios administrativos.
internal:view / manage	Acceso a consola tecnica; acciones futuras de retry/requeue requieren ademas autorizacion especifica, idempotencia y audit.

- Los nombres anteriores son una taxonomia inicial; cada modulo posterior puede agregar capacidades mas precisas sin cambiar el principio de autorizacion por accion.
- Los roles son conjuntos administrables de capacidades. Se pueden definir perfiles como Administrador, Operaciones, Administracion o Tecnico, pero el backend decide por permiso efectivo y no por la mera presencia de una etiqueta de rol.
- Un usuario autenticado con permiso puede abrir un link compartido hacia una seccion comun aunque no haya creado el recurso ni recibido el link desde la aplicacion.
- Un usuario autenticado sin permiso recibe una respuesta de acceso denegado (**403** o equivalente) sin filtrar datos. Un usuario no autenticado recibe **401** en API y redirect a login en la experiencia web.
- La navegacion puede ocultar menus y acciones no autorizadas, pero las mismas reglas se vuelven a verificar en backend.
- El principio de minimo privilegio aplica especialmente a configuracion, Internal / Technical Console, jobs, outbox, integraciones, retries y futuras acciones administrativas.

D. Identidad, autoria y auditoria

- La identidad de Supabase Auth se vincula al usuario interno de Agro Ops mediante un identificador estable; los datos de dominio no usan el email como clave primaria de autorizacion.
- Las operaciones que creen entidades registran autoria cuando corresponda, por ejemplo **created_by** y timestamps; las modificaciones relevantes se explican mediante audit_events segun la Etapa 2.
- Audit conserva actor, fecha/hora, accion, entidad/referencia y before/after cuando corresponda. La autoria no se deriva del dispositivo, URL ni cliente que envio el request.
- Usuarios deshabilitados o sin sesion valida no pueden continuar operando aunque conserven un link antiguo.
- No se modifica la politica de no borrado destructivo: identidad y permisos se integran con las reglas existentes de anulacion, reversibilidad e historia.

EJEMPLO OPERATIVO

Juan crea una labor y **Pedro** la corrige. Ambos trabajan sobre el mismo dato de la empresa si sus capacidades lo permiten. La aplicacion conserva quien creo la operacion y el audit registra quien realizo la correccion. No se crean copias privadas de la labor para cada usuario.

ACTUALIZACION v4.4 - SEGURIDAD Y PRUEBAS

E. Seguridad de la Internal / Technical Console

- Durante Etapa 1, /internal exige al menos una sesion autenticada real.
- En Etapa 2, /internal y sus endpoints requieren capacidades tecnicas/administrativas especificas; la consola no queda accesible solo por conocer la URL.
- La regla de no exposicion de secretos de v4.3 se mantiene integra y se aplica tambien a respuestas 401/403, logs del navegador y mensajes de error.

F. Pruebas y validaciones obligatorias

- E2E Etapa 1: abrir un deep link privado sin sesion -> redirect a /login -> autenticar con Supabase -> volver al destino original -> System Status funciona con API/DB/Worker reales.
- Abrir una ruta privada en una sesion nueva/incognito no muestra contenido protegido antes del login.
- Una sesion invalida o vencida no puede consumir endpoints privados aunque se llame directamente a Railway.
- No existe flujo publico de auto-registro. Una cuenta no habilitada por el mecanismo administrativo no obtiene acceso.
- El frontend adjunta el token correcto y el backend rechaza requests privadas sin token, con token malformado o con identidad no valida.
- Los endpoints declarados publicos devuelven solo informacion minima no sensible; ningun endpoint interno queda publico accidentalmente.
- Etapa 2: usuario autenticado con capacidad requerida puede acceder a datos compartidos de la empresa independientemente de quien creo el registro o compartio el link.
- Etapa 2: usuario autenticado sin capacidad requerida recibe acceso denegado y no obtiene payload parcial ni metadata sensible.
- Manipular el frontend, escribir la URL manualmente o llamar la API directamente no permite saltar permisos.
- Logout y expiracion de sesion eliminan el acceso a nuevas requests privadas.
- Audit atribuye exactamente el actor autenticado a creaciones, modificaciones, anulaciones y acciones tecnicas cubiertas por el core.
- Tests de permisos cubren al menos limites entre view/create/edit/manage y casos sensibles de configuracion e Internal Console.

ACTUALIZACION v4.4 - CRITERIOS DE SALIDA Y SECCIONES AMPLIADAS

G. Criterios de salida ampliados

ETAPA 1 - WALKING SKELETON

Ademas de todos los criterios ya existentes, staging demuestra autenticacion real: un usuario habilitado abre Vercel, inicia sesion, accede a la aplicacion privada y consulta System Status; un visitante anonimo no accede a contenido interno. El backend protege endpoints privados con identidad valida. El E2E original web -> autenticar -> system status -> API/DB/worker OK queda explicitamente satisfecho.

ETAPA 2 - NUCLEO TRANSVERSAL

Ademas de todos los criterios ya existentes, existen permisos efectivos por seccion/accion aplicados en Rust, roles como agrupadores administrables, atribucion de actor y audit. Un usuario autenticado puede usar datos comunes de la empresa segun sus capacidades, sin ownership artificial por usuario.

H. Secciones del Plan v4.3 que quedan ampliadas

- **2. Principios no negociables:** se agrega aplicacion privada por defecto, identidad individual, datos compartidos de empresa, no auto-registro publico y backend como barrera de autorizacion.
- **3. Arquitectura objetivo y componentes:** Supabase Auth provee identidad/sesion; Rust valida autenticacion y aplica autorizacion efectiva. Next.js gestiona UX de login, redirects y navegacion condicionada.
- **4. Entornos y deployment:** staging/production usan proyectos y credenciales de Auth separadas; secretos privilegiados no llegan al cliente.
- **5. GitHub, CI/CD y sistema de pruebas:** se agregan tests de deep links privados, sesion, bypass directo de API, permisos y no exposicion de datos.
- **8. Etapa 1 - Walking Skeleton:** autenticacion minima real incluye login/logout/sesion, redirect desde rutas privadas, token hacia backend y proteccion de endpoints internos.
- **9. Etapa 2 - Nucleo transversal:** Organization/usuarios/roles/permisos se concretan como datos compartidos de empresa y capacidades por seccion/accion; audit atribuye actor.
- **23. Matriz de pruebas / Seguridad:** se amplian API auth tests con autorizacion por capacidades y bypass directo.
- **24. ADR:** esta precision no cambia los componentes acordados y no requiere un ADR nuevo; un cambio futuro hacia multi-tenant real, proveedor de identidad distinto o modelo estructural de autorizacion si debera evaluarse como decision arquitectonica.

I. Roadmap y regla de compatibilidad

El orden de ejecucion continua sin cambios: **0 -> 1 -> 1.5 -> 2 -> 3 ... -> 14**. Etapa 1 implementa autenticacion real y privacidad basica; Etapa 2 materializa autorizacion fina, roles, Organization y audit. Ningun requerimiento anterior se elimina o se considera reemplazado por esta actualizacion.

REGLA DE PRIORIDAD

La privacidad y la identidad son condiciones de seguridad del producto, no una nueva vertical funcional. Se agregan a las etapas que ya las necesitaban y no desplazan staging, HTTP/2+, Postgres/PostGIS, SQLx, migrations, CI, System Status, heartbeat, DER, audit, outbox, idempotencia, ARCA, Finnegan's ni las verticales operativas.

ACTUALIZACION v4.5 - CONTEXTO PYME AGROPECUARIA ARGENTINA

CARACTER DE LA ACTUALIZACION

La version 4.5 conserva integramente el Plan Integral v4.4 y agrega dos precisiones aditivas: (1) el producto se define expresamente para una PyME agropecuaria argentina que opera agricultura y ganaderia dentro de una misma organizacion, y (2) se incorpora como requisito futuro la integracion con el Baston de Lectura XRS2i para la carga de datos de animales. No elimina, sustituye, posterga ni reduce la prioridad de ningun requerimiento, etapa, prueba, integracion, ADR o criterio de salida existente.

A. Contexto funcional y localizacion

Principio	Regla v4.5
PyME agropecuaria argentina	Agro Ops V1 es una aplicacion privada para una unica empresa agropecuaria argentina. Agricultura y ganaderia son actividades de la misma organizacion y comparten la infraestructura transversal.
Datos de la empresa	Los datos operativos pertenecen a la organizacion. No se crean silos por usuario, por establecimiento ni por actividad salvo limites funcionales expresamente definidos.
Idioma funcional	Interfaz, mensajes, permisos visibles, documentacion funcional, ayuda y ejemplos de negocio se expresan en espanol y con vocabulario natural del agro argentino.
Convenciones tecnicas	Se mantienen en ingles nombres propios de tecnologias, protocolos, librerias y conceptos tecnicos cuando eso mejora interoperabilidad o evita ambigüedad. Los codigos internos se documentan de forma estable y no sustituyen las etiquetas visibles en espanol.
Unidades y formatos	La interfaz usa formatos adecuados a Argentina y unidades habituales del negocio: ha, kg, t, L, km y h, preservando las reglas Decimal/NUMERIC ya vigentes para datos criticos.

B. Autorizacion adaptada al negocio

- Se conserva integramente el modelo de autorizacion por capacidades de v4.4: el backend Rust decide por permiso efectivo y no por el nombre del rol.
- Los ejemplos de roles y capacidades deben usar terminologia comprensible para la empresa, por ejemplo Agricultura, Ganaderia, Stock, Maquinaria, Comercializacion, Configuracion y Consola tecnica.
- Los ejemplos de codigos de capacidades definidos en v4.4 conservan su semantica. Al materializar Etapa 2 puede adoptarse una taxonomia canonica equivalente en espanol, documentando la equivalencia y sin perder ninguna capacidad preexistente.
- No se diseña V1 como SaaS multiempresa. Organization se conserva como contexto y limite de datos, sin introducir complejidad multi-tenant que el negocio no necesita hoy.

REGLA DE COMPATIBILIDAD

Toda precision de idioma, terminologia o contexto empresarial es aditiva. Ningun requisito de v4.4 se considera reemplazado. La arquitectura, las etapas, las integraciones, la politica de no-delete, Decimal, auditoria, idempotencia, outbox, CI, staging, ARCA y Finnegans mantienen su alcance y prioridad.

ACTUALIZACION v4.5 - INTEGRACION CON BASTON DE LECTURA XRS2i

C. Nuevo requisito de captura ganadera

Agro Ops debera poder incorporar lecturas de identificacion electronica (EID/IDE) realizadas con un Baston de Lectura XRS2i (Tru-Test/Datamars) y utilizarlas como entrada controlada para la operacion ganadera. El dispositivo no se convierte en fuente de verdad del animal: aporta lecturas y sesiones de captura; Agro Ops conserva la identidad operativa del animal, su historia, movimientos y asociaciones.

Capacidades tecnicas del XRS2i relevantes para el diseño

- Modo independiente: el lector puede guardar las etiquetas EID escaneadas en su memoria dentro de sesiones y transferirlas posteriormente.
- Modo conectado: puede transmitir cada etiqueta escaneada a otro dispositivo mediante Bluetooth.
- Puede transferir sesiones a una PC por USB usando Data Link y a dispositivos moviles mediante las aplicaciones soportadas por Datamars.
- Dispone de modo HID (Human Interface Device), que permite transmitir el EID como entrada equivalente a teclado hacia una aplicacion que no tenga una integracion especifica con el lector.
- El lector puede registrar, ademas del EID, informacion como identificacion visual (VID), sexo, raza, fecha de nacimiento, condicion corporal y campos personalizados; Agro Ops solo importara los campos que tengan semantica de dominio aprobada.

D. Estrategia de integracion

Modo	Alcance en Agro Ops	Prioridad
1. Importacion de sesion	El usuario obtiene el archivo de sesion desde XRS2i/Data Link o la app soportada y lo carga en Agro Ops. El backend valida, normaliza, previsualiza y confirma la importacion con idempotencia.	OBLIGATORIO V1
2. Captura en vivo por HID	POC para emparejar el XRS2i con la PC/tablet y recibir el EID en una pantalla de captura de Agro Ops como entrada de teclado. Debe validarse con los dispositivos/navegadores reales de la empresa.	POC ETAPA 7
3. Integracion vendor/cloud	Solo se incorpora si existe un mecanismo oficial, estable y justificable de Datamars que aporte valor. Queda detras de adapter y no es dependencia obligatoria de V1.	OPCIONAL FUTURA

DECISION DE ARQUITECTURA

V1 no dependera de Web Bluetooth ni de un protocolo propietario del XRS2i para poder operar. La ruta base sera importar sesiones y la ruta de captura inmediata se validara primero mediante HID. Cualquier integracion directa adicional quedara detras de un adapter reemplazable.

E. Operacion con conectividad limitada

El diseño online-first se conserva. En un corral o manga sin conectividad, el XRS2i puede trabajar de manera independiente, almacenar una sesion local y transferirla mas tarde. Esto permite capturar animales en campo sin convertir Agro Ops en una aplicacion offline-first completa.

Las lecturas capturadas fuera de linea no se consideran aplicadas al dominio hasta que Agro Ops valide y confirme su importacion.

ACTUALIZACION v4.5 - MODELO DE DATOS PARA LECTURAS EID

F. Ampliacion conceptual del dominio Ganaderia

La Etapa 1.5 permanece cerrada y valida. Este nuevo requerimiento agrega un addendum conceptual que debe incorporarse al DER maestro antes de materializar la parte correspondiente de Ganaderia. No se crean estas tablas durante Etapa 2.

Concepto	Responsabilidad
Animal	Entidad operativa ganadera. Conserva su identidad interna UUID y su historia independientemente del dispositivo que lo lea.
IdentificacionAnimal	Asocia al Animal una identificacion electronica EID/IDE y, cuando corresponda, una identificacion visual. Permite preservar historia de reemplazos o correcciones sin usar el EID como PK interna.
DispositivoLectura	Registra metadata no sensible del equipo utilizado, por ejemplo tipo/modelo XRS2i y un identificador/serie cuando este disponible y sea util. No contiene secretos.
SesionLecturaGanado	Representa una sesion de trabajo: fecha, establecimiento/contexto cuando corresponda, origen, dispositivo y estado de importacion.
LecturaEID	Registro inmutable de una lectura recibida: valor raw, valor normalizado, timestamp disponible, orden dentro de la sesion, campos asociados y resultado de vinculacion.
ImportacionLecturas	Control de archivo/lote recibido, hash/fingerprint, usuario importador, parser/version y resultado. Permite idempotencia, reintento seguro y trazabilidad.

G. Invariantes de identificacion y trazabilidad

- El UUID interno del Animal es la identidad primaria de Agro Ops. El EID no reemplaza el ID interno.
- Se conserva tanto el valor recibido como su representacion normalizada. El formato de lectura no debe destruir informacion necesaria para diagnostico o reimportacion.
- Un mismo EID activo no puede identificar simultaneamente a dos animales distintos salvo un flujo de correccion explicito, autorizado y auditado.
- Importar dos veces el mismo archivo o la misma sesion no debe duplicar animales, identificaciones ni eventos. Se reutilizan las primitivas de idempotencia de Etapa 2.
- Una lectura desconocida no crea automaticamente un animal definitivo. Puede quedar pendiente para vincular, dar de alta o descartar con motivo auditado.
- No se borran lecturas confirmadas para "corregir" errores. Se conserva la captura original y se audita la vinculacion, desvinculacion, reclasificacion o anulacion correspondiente.
- Los campos adicionales del XRS2i no se vuelcan como JSON generico sin semantica. Raza, sexo, fecha de nacimiento, condicion corporal u otros datos se mapean solo cuando el dominio los haya definido.
- La configuracion de permitir/no permitir duplicados en el lector no sustituye la idempotencia del backend; Agro Ops debe ser robusto ante duplicados igualmente.

Flujo conceptual

XRS2i -> Sesion/archivo o HID -> Adapter de captura -> Validacion/normalizacion -> Previsualizacion -> Confirmacion transaccional -> LecturaEID + vinculo a Animal + AuditEvent

FUENTE DE VERDAD

El XRS2i es fuente de la lectura capturada. Agro Ops es fuente de verdad de la identidad operativa del animal y de la interpretacion de esa lectura dentro del negocio. Finnegans no recibe lecturas EID por defecto y no se genera ningun efecto contable por escanear un animal.

ACTUALIZACION v4.5 - ETAPA 7 GANADERIA AMPLIADA

H. Etapa 7 - Ganaderia: incorporacion del XRS2i

Se conserva integramente el alcance de Ganaderia definido en v4.4. Se agregan los siguientes flujos de captura y vinculacion sin eliminar movimientos, categorias, rodeos, pesadas, compras/ventas, costos ni conciliaciones ya previstas.

Flujo operativo esperado en Agro Ops - ampliacion

- Registrar y administrar la identificacion electronica del animal mediante EID/IDE y su identificacion visual cuando corresponda.
- Crear una pantalla de "Sesiones de lectura" o "Carga desde XRS2i" para importar, revisar y confirmar capturas.
- Permitir importar una sesion desde archivo con vista previa: leídos, reconocidos, nuevos/no vinculados, duplicados, invalidos y conflictos.
- Permitir vincular una lectura desconocida a un animal existente o iniciar el alta correspondiente segun permisos y reglas del dominio.
- Realizar un POC de captura en vivo mediante modo HID del XRS2i. La confirmacion de la operacion sigue ocurriendo en Agro Ops; el escaneo por si solo no modifica silenciosamente existencias.
- Cuando una lectura se use dentro de un traslado, pesada, cambio de categoria, tratamiento u otro evento futuro, el caso de uso debe declarar explicitamente que efecto genera; leer el EID no equivale automaticamente a ejecutar el evento.
- Si un indicador de pesaje u otra fuente aporta peso junto al EID, ese dato se integra por el contrato de captura definido para pesadas. El XRS2i por si mismo se trata como lector de identificacion, no como fuente de peso.

Permisos de negocio

La taxonomia de autorizacion de Etapa 2 se amplia en Ganaderia solamente cuando la funcionalidad se materialice. Como minimo se evaluarán capacidades equivalentes a **ganaderia:importar_lecturas**, **ganaderia:vincular_identificacion** y **ganaderia:administrar_dispositivos**. Los nombres definitivos deben seguir la convencion canonica aprobada en Etapa 2.

Rol de Finnegan, ARCA y sistemas externos

- Las lecturas del XRS2i son informacion operativa de Agro Ops. No crean facturas, asientos, ingresos, egresos ni documentos ficticios en Finnegan.
- Una compra/venta fiscal de hacienda sigue naciendo o referenciandose segun el circuito Finnegan ya definido; el EID puede ayudar a vincular la consecuencia fisica, pero no reemplaza el documento fiscal.
- ARCA no participa en este flujo.
- No se asume integracion automatica con SENASA u otro sistema oficial de trazabilidad animal. Si se requiere, se incorporara como integracion externa separada, con POC y reglas de ownership propias, sin contaminar el adapter XRS2i.

Criterio de salida ampliado de Etapa 7

CRITERIO DE SALIDA

Ademas de los criterios ganaderos preexistentes, una sesion real del XRS2i puede incorporarse a staging sin duplicados, con EID normalizados, conflictos visibles, usuario importador identificado, trazabilidad completa y vinculacion controlada a animales. El POC HID se valida con al menos uno de los dispositivos cliente reales de la empresa o se documenta la limitacion observada.

ACTUALIZACION v4.5 - PRUEBAS, ROADMAP Y SECCIONES AMPLIADAS

I. Pruebas obligatorias de la integracion XRS2i

Nivel	Prueba esperada
Parser / fixtures	Archivos/sesiones anonimizadas con EID validos, invalidos, repetidos, campos faltantes, orden distinto y campos adicionales. El parser conserva raw + normalizado.
DB integration	Unicidad de identificacion activa, historia de reemplazos/correcciones, FK de sesion/dispositivo/animal, no-delete destructivo.
Idempotencia	Mismo archivo o sesion importado N veces produce una sola consecuencia. Retry tras fallo parcial no duplica vinculos ni eventos.
API / autorizacion	Usuario sin permiso no importa ni vincula identificaciones. 401/403 no filtran datos de animales ni metadata sensible.
Audit	Importacion, confirmacion, vinculacion, desvinculacion y correcciones relevantes registran actor y contexto.
UI / E2E	Archivo -> preview -> resolver pendientes -> confirmar -> ficha del animal actualizada. Simulacion de entrada HID para regresion automatica.
POC hardware	Prueba manual controlada con XRS2i real: transferencia de sesion y, cuando aplique, captura HID. No se exige hardware fisico en CI de cada push.

J. Etapas y secciones del Plan v4.4 que quedan ampliadas

- **1 / 2. Resumen y principios:** se explicita PyME agropecuaria argentina, una sola organizacion V1, idioma funcional espanol y vocabulario local.
- **3. Arquitectura:** XRS2i se trata como dispositivo externo detras de un adapter de captura; el dominio Ganaderia no depende de Bluetooth, Data Link ni Datamars.
- **5. CI/CD y pruebas:** se agregan fixtures/parser/idempotencia y pruebas simuladas; el hardware real se valida en POC/milestone.
- **8A. Etapa 1.5 / DER:** se agrega un addendum conceptual para identificacion animal, sesiones, lecturas, dispositivo e importaciones antes de materializar Ganaderia. No reabre ni invalida la salida ya aprobada de Etapa 1.5.
- **9. Etapa 2:** no implementa XRS2i. Sus primitivas de permisos, audit, idempotencia, Documents y jobs quedan disponibles para reutilizarse en la integracion posterior.
- **14. Etapa 7 - Ganaderia:** se convierte en la etapa principal de implementacion de XRS2i.
- **20. Etapa 13 - Conciliacion/alertas:** puede alertar lecturas/importaciones pendientes, conflictos de EID o sesiones con errores sin duplicar la pantalla operativa de Ganaderia.
- **21. Etapa 14 - Piloto/cutover:** el piloto ganadero incluye una prueba de campo con el XRS2i y procedimiento documentado de importacion/recuperacion.
- **23. Matriz de pruebas:** Ganaderia incorpora parser, idempotencia de importacion, hardware POC y trazabilidad de identificacion.

K. Roadmap

El orden general continua sin cambios: **0 -> 1 -> 1.5 -> 2 -> 3 ... -> 7 -> ... -> 14**. No se crea una etapa independiente para el XRS2i. La funcionalidad se implementa dentro de Ganaderia y reutiliza el core transversal construido en Etapa 2.

REGLA DE PRIORIDAD

La integracion XRS2i y la localizacion argentina son requisitos aditivos. No eliminan, sustituyen, postergan ni reducen ningun requerimiento preexistente. En particular no desplazan Etapa 2, Territorio, Stock, Agricultura, Maquinaria, ARCA, Finnegan, costos, conciliacion, audit, outbox, idempotencia, CI, staging ni los criterios de salida ya definidos.

Fuentes tecnicas de referencia para XRS2i (consultadas para esta actualizacion)

• Tru-Test Datamars LATAM - Guias de usuario: XRS2i Stick Reader Quickstart Guide. • Datamars Livestock Knowledge Base - coleccion XRS2i/XRS2; transferencia de sesiones a Datamars Pro; conectividad Bluetooth y soporte del equipo. • XRS2i Quickstart/User Guide - modos independiente/conectado, transferencia Data Link, Bluetooth, HID, formatos de tag y campos de datos.

ACTUALIZACION v4.6 - PANEL DE CONTROL ADMINISTRATIVO

CARACTER DE LA ACTUALIZACION

La version 4.6 conserva integramente el Plan Integral v4.5 y agrega una capacidad transversal de administracion dentro de Agro Ops: un **Panel de Control Administrativo**. Su objetivo es que las tareas normales de configuracion, seguridad y gobierno funcional puedan realizarse desde la aplicacion por usuarios autorizados, sin depender de editar PostgreSQL, copiar UUID, ejecutar comandos, ingresar a Railway o manipular configuracion interna. No elimina ni sustituye la Internal / Technical Console, no convierte al administrador en desarrollador y no habilita cambios de codigo o de funcionalidad.

A. Principios del Panel de Control Administrativo

Principio	Regla v4.6
Administracion autoservicio	Toda tarea administrativa recurrente que sea parte del producto debe disponer de una operacion soportada desde Agro Ops. El acceso directo a base de datos o CLI queda reservado a bootstrap, recuperacion o soporte excepcional.
Backend autoritativo	Next.js presenta la experiencia administrativa; Rust autentica, autoriza y ejecuta cada operacion. Ningun boton, ruta o estado del frontend sustituye la verificacion del permiso efectivo.
No es acceso root	El panel administra datos y configuraciones modeladas por el producto. No permite ejecutar SQL, modificar codigo, editar variables de entorno, cambiar secretos, instalar componentes ni alterar infraestructura.
Permisos por capacidad	configuracion:ver habilita consulta donde corresponda y configuracion:administrar habilita cambios administrativos generales. Consola tecnica conserva sus permisos propios. Los roles no se usan como condicion de seguridad.
Historia y auditabilidad	Deshabilitaciones, cambios de rol, permisos y configuraciones sensibles preservan historia. Una vez materializado audit_events, toda accion administrativa relevante registra actor, fecha, accion y cambio.
Configuracion con dueño	No se crea una tabla generica de claves/valores para todo el sistema. Cada configuracion vive en el modelo normalizado del dominio que la posee; el panel solo centraliza su administracion.

B. Estructura prevista

- **/configuracion** - inicio del Panel de Control Administrativo y resumen de pendientes/configuracion disponible.
- **/configuracion/usuarios** - altas/invitaciones, habilitacion, deshabilitacion, estado y asignacion de roles.
- **/configuracion/roles** - roles de la organizacion y composicion mediante permisos canonicos.
- **/configuracion/organizacion** - datos administrativos de la organizacion que hayan sido materializados como configurables; IDs internos e invariantes estructurales no se editan.
- **/configuracion/seguridad** - controles administrativos soportados de identidad/acceso, sin exponer secretos ni credenciales.
- **/configuracion/... por dominio** - catalogos y parametros aparecen solamente cuando la etapa que los necesita materializa su modelo.

REGLA DE ALCANCE

El objetivo es eliminar la administracion cotidiana por backend, no esconder operaciones de ingenieria dentro de una UI. Si una accion exige cambiar codigo, schema, secreto, deployment o arquitectura, permanece fuera del Panel de Control y sigue el flujo tecnico normal.

ACTUALIZACION v4.6 - USUARIOS, ROLES Y SEGURIDAD

C. Administracion de usuarios

La administracion normal de personas se realiza desde Agro Ops. Supabase Auth conserva la identidad y la sesion; PostgreSQL conserva si la persona esta habilitada en Agro Ops y que capacidades posee.

- Listar usuarios de la organizacion con nombre, estado, roles vigentes y estado administrativo de la identidad cuando pueda conocerse de forma segura.
- Crear o invitar un usuario ingresando como minimo nombre completo, correo electronico y roles iniciales. El administrador no copia ni ingresa manualmente el UUID de Supabase.
- Rust utiliza una integracion server-side con Supabase Auth Admin para crear o resolver la cuenta y obtiene el sujeto UUID estable; luego crea el Usuario, la IdentidadAutenticacionExterna y sus UsuarioRol dentro de una transaccion PostgreSQL.
- El correo electronico es dato de administracion/entrega de invitacion; nunca es la clave de autorizacion. La vinculacion efectiva sigue siendo (proveedor = supabase, sujeto_proveedor = UUID).
- Habilitar o deshabilitar un Usuario. usuarios.activo = false elimina acceso de Agro Ops desde el request siguiente aunque la sesion Supabase siga existiendo.
- Cambiar roles cerrando la vigencia de asignaciones anteriores y creando las nuevas cuando corresponda. No se borra historia para simular que una asignacion nunca existio.
- Permitir reenviar una invitacion o iniciar un flujo soportado de recuperacion de acceso cuando Supabase lo permita, sin que el administrador vea o defina contraseñas en texto plano.

D. Roles y permisos

- Listar roles de la organizacion, su estado, cantidad de usuarios vigentes y permisos efectivos asociados.
- Crear roles administrables de la organizacion y habilitarlos/deshabilitarlos cuando corresponda.
- Asignar o quitar permisos canonicos a un rol mediante roles_permisos y sus intervalos de vigencia.
- Los permisos son capacidades definidas por el software y sus migrations. El administrador puede asignarlos, pero **no crear codigos arbitrarios, cambiar su semantica ni redefinir una funcionalidad desde la UI**.
- La autorizacion nunca compara roles.nombre. Renombrar un rol, si se habilita, no altera por si mismo las capacidades efectivas.
- No se agrega usuarios_permisos: se conserva Usuario -> UsuarioRol -> Rol -> RolPermiso -> Permiso.

E. Salvaguardas administrativas

Salvaguarda	Comportamiento requerido
Ultimo administrador	No debe ser posible dejar a la organizacion sin ningun usuario activo con configuracion:administrar por una accion ordinaria. Deshabilitarse o quitarse el ultimo acceso administrativo debe bloquearse o exigir un flujo de recuperacion explicito.
Cuenta Supabase sin alta Agro Ops	Autenticarse en Supabase no concede acceso. Si no existe un vinculo interno vigente y un Usuario activo, los endpoints privados responden 403.
Fallo parcial de alta	Si Supabase crea/invita la cuenta pero la transaccion interna falla, la cuenta queda sin acceso a Agro Ops. Reintentar debe ser seguro y no duplicar identidades, usuarios ni asignaciones.
Identidad ya vinculada	Un sujeto Supabase asociado a otro Usuario produce conflicto; nunca se religa silenciosamente.
Desactivacion	El acceso se corta en la autorizacion interna. No se depende exclusivamente de eliminar o suspender la cuenta en Supabase.

BOOTSTRAP INICIAL

El primer administrador sigue requiriendo un mecanismo tecnico unico para romper la dependencia circular. Puede usar el sujeto UUID de Supabase una sola vez y debe recibir al menos configuracion:administrar. Si tambien administrara la consola tecnica, recibe consola_tecnica:ver. Luego, los usuarios normales se administran desde Agro Ops sin UUID ni CLI.

ACTUALIZACION v4.6 - DASHBOARD Y CONFIGURACION ADMINISTRATIVA

F. Dashboard del administrador

La portada /configuracion funciona como tablero de control administrativo. No mantiene un segundo estado paralelo: resume informacion derivada de las fuentes reales y enlaza a la accion correspondiente.

Area	Contenido del dashboard
Accesos	Usuarios activos/inactivos, invitaciones o altas pendientes cuando esa informacion este disponible, roles vigentes y alertas de configuracion de acceso.
Roles y permisos	Cantidad de roles activos, roles sin usuarios, permisos canonicos disponibles y advertencias administrativas relevantes.
Organizacion	Datos configurables de la organizacion y avisos por parametros obligatorios faltantes cuando exista ese modelo.
Configuracion de dominios	Accesos a catalogos/parametros que cada modulo haya materializado. El panel no duplica pantallas operativas ni crea datos futuros.
Estado del sistema	Resumen o enlace a /internal/system-status solo para quien posea consola_tecnica:ver. Un administrador funcional sin permiso tecnico no obtiene informacion interna por el solo hecho de administrar usuarios.
Integraciones	Cuando existan ARCA/Finnegans, puede mostrar estado administrativo no sensible y enlaces a mapeos/configuracion permitida. Diagnostico profundo, retries y secretos siguen sus permisos tecnicos especificos.
Auditoria	Acceso de consulta a cambios administrativos y eventos auditables cuando audit_events este materializado, con filtros por actor, fecha y entidad segun el permiso definido.

G. Relacion con la Internal / Technical Console

El Panel de Control Administrativo y la Internal Console son superficies diferentes que pueden enlazarse, pero no se fusionan conceptualmente:

Superficie	Responsabilidad	Permiso base
Panel de Control	Administracion de usuarios, roles, configuracion funcional y maestros permitidos.	configuracion:ver / configuracion:administrar
Internal Console	Diagnostico tecnico: System Status, jobs, outbox, integraciones, retries y observabilidad.	consola_tecnica:ver / consola_tecnica:administrar y permisos futuros especificos

H. Configuracion administrativa que se incorpora por etapa

Etapas	Extension administrativa
2 - Nucleo transversal	Panel /configuracion, usuarios, roles/permisos, bootstrap inicial, gate web por capacidades, resumen administrativo y base para auditoria de cambios.
3 - Territorio	Solo maestros/parametros realmente definidos como configurables. Geometrias, campañas y UOP continuan en sus flujos operativos; no se convierten automaticamente en "settings".
4 - Stock	Maestros de productos, unidades y depositos cuando su administracion se materialice; politicas sensibles de stock requieren permiso explicito y reglas de dominio.
5 - Agricultura	Catalogos de actividades/labores y parametros de negocio que el dominio declare administrables.
6 - Maquinaria	Catalogos y parametros de equipos/mantenimiento cuando corresponda, sin mezclar altas/eventos operativos con configuracion global.
7 - Ganaderia	Categorias/catalogos administrativos y DispositivoLectura; configuracion XRS2i no expone secretos ni convierte la lectura en efecto automatico.
8-9 - Granos/Comercial	Maestros y parametros comerciales que se aprueben, manteniendo documentos y movimientos en sus pantallas operativas.
10-11 - ARCA/Finnegans	Mapeos y opciones no sensibles aprobadas. Certificados, private keys, tokens, passwords y connection strings permanecen fuera de la UI.
13 - Conciliacion	Tolerancias, responsables o reglas de alertas solo si se modelan explicitamente; no existe un JSON generico de configuracion.

ACTUALIZACION v4.6 - ARQUITECTURA DEL PANEL ADMINISTRATIVO

I. Flujo tecnico obligatorio

Toda modificacion del Panel de Control sigue la misma frontera de seguridad y ownership del resto de Agro Ops:

FLUJO

Navegador / Next.js -> access token Supabase -> Rust API -> AuthorizationContext -> require_permission(...) -> caso de uso administrativo -> PostgreSQL / adapter externo autorizado

- El navegador nunca escribe directamente tablas de autorizacion ni utiliza service-role/admin credentials.
- Next.js puede ocultar o bloquear navegacion segun capacidades, pero Rust vuelve a exigir el permiso en cada endpoint administrativo.
- Las operaciones multi-tabla de Agro Ops se confirman en una transaccion PostgreSQL. Los efectos externos, como crear/invitar en Supabase, se diseñan para fallo seguro e idempotencia de reintento.
- Las respuestas 401/403/409/5xx no deben filtrar metadata de identidades, roles de terceros, secretos o detalles internos innecesarios.

J. Integracion privilegiada con Supabase Auth

- El backend incorpora un adapter/puerto explicito para las operaciones administrativas de Supabase Auth que el flujo de usuarios requiera.
- La credencial privilegiada se configura exclusivamente en el runtime de Rust y se redacta de Debug, errores y logs. Nunca usa prefijo NEXT_PUBLIC ni se envia a Vercel/browser.
- Supabase Auth es dueño de email verificado, invitacion, password y sesion. Agro Ops no almacena passwords ni usa email para autorizar.
- Una cuenta externa no habilitada internamente sigue sin acceso. Esta propiedad es la barrera de seguridad ante altas parciales o cuentas creadas fuera de Agro Ops.

K. Limites: que NO puede cambiar el administrador desde el panel

Limite	Regla
Codigo y funcionalidades	No puede activar codigo inexistente, cambiar reglas de dominio, modificar endpoints, migrations, dependencias ni arquitectura.
Permisos canonicos	Puede asignarlos a roles; no puede inventar codigos de permisos ni cambiar la semantica que Rust interpreta.
Base de datos	No hay editor SQL, tabla generica ni consola de datos arbitraria.
Infraestructura	No administra Railway/Vercel/Supabase projects, dominios, builds, Docker ni variables de entorno desde Agro Ops.
Secretos	No ve ni edita service-role keys, certificados privados, tokens, passwords o connection strings. La rotacion sigue el procedimiento de infraestructura.
Historia confirmada	No borra historial para corregir. Se usan deshabilitacion, cierre de vigencia, anulacion o correccion auditada segun el tipo de dato.

CRITERIO DE DISEÑO

La interfaz administrativa debe hacer modificable aquello que pertenece al negocio/configuracion del producto y dejar fuera aquello que pertenece a ingenieria o infraestructura. "No tener que tocar el backend" significa que la operacion administrativa normal dispone de un caso de uso soportado, no que el backend deje de existir como barrera de seguridad.

ACTUALIZACION v4.6 - IMPLEMENTACION, PRUEBAS Y CRITERIO DE SALIDA

L. Implementacion incremental

Bloque	Resultado
Slice 2.4 - Gate web	Exponer el contexto minimo del usuario autenticado y evitar que /internal u otras rutas protegidas se rendericen por mera autenticacion. Next.js usa capacidades; Rust conserva la barrera real.
Slice 2.5 - Panel administrativo base	Crear /configuracion, /configuracion/usuarios y /configuracion/roles; alta/invitacion de usuarios mediante Supabase Admin server-side; habilitar/deshabilitar; asignar roles; administrar composicion de roles.
Etapas 2 - Audit transversal	Cuando audit_events se materialice, integrar las acciones administrativas sensibles al mismo mecanismo. No crear un sistema de audit paralelo dentro del panel.
Etapas posteriores	Cada dominio incorpora sus propios catalogos/parametros administrables cuando materializa su modelo. /configuracion los centraliza por navegacion, no duplica ownership ni crea schema anticipado.

M. Pruebas y validaciones obligatorias

- Usuario anonimo: deep link administrativo -> login; despues del login vuelve solo si su usuario interno y permisos lo autorizan.
- Cuenta Supabase valida pero sin Usuario Agro Ops habilitado -> 403 y ningun contenido administrativo.
- Usuario sin configuracion:administrar -> no puede modificar usuarios/roles desde UI ni llamando directamente a Railway.
- Administrador crea/invita usuario desde Agro Ops -> obtiene sujeto estable -> transaccion interna crea vinculo/roles -> el nuevo usuario solo accede a sus capacidades.
- Fallo de PostgreSQL despues de crear/invitar en Supabase -> la cuenta externa queda sin acceso; reintentar no duplica identidad ni asignaciones.
- Deshabilitar Usuario -> el request siguiente pierde acceso sin depender de borrar la cuenta externa.
- Quitar rol o permiso -> se cierra vigencia; los registros historicos permanecen y la autorizacion cambia en el siguiente request.
- No puede dejarse la organizacion sin administrador activo por una operacion ordinaria.
- El dashboard no muestra System Status a quien no tenga consola_tecnica:ver y no duplica estados calculados.
- Ningun HTML, JSON, log de navegador o error expone credenciales privilegiadas de Supabase ni otros secretos.
- Los permisos canonicos no pueden crearse/redefinirse desde la UI; solo asignarse conforme al modelo aprobado.

N. Criterio de salida ampliado de Etapa 2

CRITERIO DE SALIDA

Ademas de todos los criterios preexistentes de Etapa 2, un administrador autorizado puede realizar desde Agro Ops las tareas normales de administracion de acceso -alta/invitacion, habilitacion/deshabilitacion, roles y permisos- y consultar el resumen administrativo sin usar Supabase Dashboard, SQL, Railway ni comandos manuales. El primer bootstrap y la gestion de secretos/infraestructura quedan como excepciones tecnicas documentadas. El frontend respeta capacidades y Rust conserva la autorizacion efectiva en cada endpoint.

O. Secciones del Plan v4.5 que quedan ampliadas

- **2. Principios no negociables:** se agrega administracion autoservicio, limites de la UI administrativa, historia y separacion entre configuracion funcional e ingenieria.
- **3. Arquitectura objetivo:** Next.js incorpora Panel de Control; Rust mantiene casos de uso y autorizacion; Supabase Auth Admin queda detras de adapter privilegiado server-side.
- **5. CI/CD y pruebas:** se agregan pruebas de administracion, fallo parcial, bypass directo, ultimo administrador, no exposicion de secretos e historia temporal.
- **9. Etapa 2 - Nucleo transversal:** usuarios/roles/permisos dejan de ser solo primitivos de backend y adquieren administracion normal desde el producto.
- **Actualizacion v4.3 - Internal Console:** se mantiene separada; el Panel de Control puede mostrar un enlace/resumen condicionado por consola_tecnica:ver.
- **Etapas 3-13:** cada dominio expone en Configuracion solamente los maestros/parametros que le pertenecen cuando sean materializados.
- **24. ADR:** esta ampliacion no cambia proveedores ni ownership y no requiere ADR por si sola; una plataforma de configuracion generica, IAM distinto o acceso administrativo a infraestructura si seria una decision arquitectonica nueva.

REGLA DE PRIORIDAD

El Panel de Control Administrativo es una capacidad transversal y aditiva. No elimina, sustituye ni posterga audit, jobs, outbox, Documents, idempotencia, Territorio, Stock, Agricultura, Maquinaria, Ganaderia/XRS2i, ARCA, Finnegans, costos, conciliacion, staging, CI ni requisitos anteriores. El roadmap general continua 0 -> 1 -> 1.5 -> 2 -> 3 ... -> 14.

ACTUALIZACION v4.7 - MODELO TERRITORIAL, RENSPA/SENASA Y JERARQUIA GEOGRAFICA

CARACTER DE LA ACTUALIZACION

La version 4.7 conserva integramente el Plan Integral v4.6 y todas las actualizaciones anteriores. Esta ampliacion formaliza la jerarquia territorial que se utilizara en Agro Ops, resuelve la fuente inicial prevista para el perimetro de los establecimientos cuando exista informacion SENASA/RENSPA y distingue ese limite de los lotes internos y de las futuras unidades operativas por campaña. No elimina, sustituye, posterga ni reduce la prioridad de ningun requerimiento, etapa, prueba, integracion, ADR o criterio de salida existente.

A. Jerarquia territorial canonica

Concepto	Regla v4.7
Organizacion	En V1 una unica empresa puede administrar multiples establecimientos/campos . Todos permanecen dentro del contexto organizacional y de autorizacion definido en Etapa 2.
Establecimiento	Representa el campo o propiedad fisica completa . Tiene UUID interno, codigo canonico, nombre, limite geografico propio y referencias externas cuando corresponda. Puede existir sin lotes cargados y puede contener cero o muchos LoteBase.
RENSPA / SENASA	Se trata como identidad o referencia externa asociada al Establecimiento , no como PK de Agro Ops ni como sinonimo de lote. El identificador externo no codifica por si mismo los vertices geograficos. Cuando la realidad requiera mas de una referencia externa para el mismo establecimiento, el modelo no debe forzar una relacion artificial 1:1.
LoteBase	Subdivison interna y relativamente estable definida por la empresa dentro de un Establecimiento. Cada lote tiene identidad y geometria propias. El perimetro SENASA/RENSPA del campo no se almacena simulando ser un LoteBase .
Campaña / UOP	La Campaña aporta el contexto temporal. Las UOP se materializan en Slice 3.2 y posteriores como subdivisiones operativas que pueden cambiar entre campañas sin modificar la historia de campañas anteriores ni la identidad del LoteBase.

DECISION DE ARQUITECTURA

La jerarquia territorial queda fijada como **Organizacion -> Establecimiento -> LoteBase -> Campaña/UOP**. El Establecimiento conserva el limite del campo completo; los LoteBase representan divisiones internas; las UOP representan divisiones operativas temporales. Ninguna de estas capas se utiliza para reemplazar a otra.

Representacion conceptual

```
Organizacion
-> Establecimiento A [RENSPA / limite completo]
    -> LoteBase A1
    -> LoteBase A2
    -> ...
-> Establecimiento B [RENSPA / limite completo]
    -> LoteBase B1
    -> ...
```

ACTUALIZACION v4.7 - GEOMETRIA, PROVENIENCIA Y REFERENCIAS EXTERNAS

B. Modelo geografico de Establecimiento y LoteBase

Elemento	Regla geografica
Establecimiento.geometria	Perimetro completo del campo. Almacenamiento canonico PostGIS geometry(MultiPolygon, 4326) . Debe ser no nulo, no vacio, valido y con SRID 4326.
LoteBase.geometria	Poligono/multipoligono de la division interna. Tambien geometry(MultiPolygon, 4326) , con las mismas validaciones basicas.
Contencion	Un LoteBase debe quedar espacialmente cubierto por su Establecimiento. El criterio admite compartir/tocar el borde del campo y rechaza geometria parcial o totalmente exterior. La regla debe permanecer protegida en PostgreSQL/PostGIS.
Indices	Establecimiento y LoteBase disponen de indices GiST apropiados para consultas espaciales y visualizacion futura.
Superficie	No se guarda una superficie editable que pueda divergir de la geometria. Las hectareas se derivan con una estrategia geografica/proyectada correcta; nunca se interpretan grados de EPSG:4326 como metros cuadrados.

C. Proveniencia de la geometria

Agro Ops debe poder distinguir de donde proviene el limite geografico de un Establecimiento. Esta metadata de procedencia es parte del dato territorial y no una configuracion generica. La implementacion inicial admite valores normalizados equivalentes a **senasa_rensa**, **manual** e **importada**.

- **senasa_rensa**: perimetro obtenido a partir de informacion georreferenciada asociada al contexto SENASA/RENSPA.
- **manual**: geometria creada o corregida deliberadamente dentro de un flujo soportado por Agro Ops.
- **importada**: geometria recibida desde un archivo o fuente cartografica externa distinta del flujo SENASA/RENSPA.
- La procedencia no reemplaza validaciones PostGIS ni convierte a la fuente externa en dueña de la identidad interna del Establecimiento.

D. RENSPA/SENASA detras de ExternalReference

Se reutiliza la primitiva transversal **ExternalReference** de Etapa 2 para vincular el UUID interno de un Establecimiento con su identificador externo SENASA/RENSPA. No se agrega una segunda abstraccion de integraciones ni se introduce dependencia del dominio Territorio hacia APIs, formatos o credenciales de SENASA.

Campo conceptual	Tratamiento
sistema_externo	senasa
external_id	Valor RENSPA/SENASA tratado como identificador opaco; no se interpreta como coordenadas.
entidad_tipo	establecimiento
entidad_id	UUID interno de Agro Ops; sigue siendo la identidad primaria del dominio.

FUENTE DE VERDAD	SENASA/RENSPA puede aportar una referencia y un perimetro georreferenciado de origen. Agro Ops conserva la identidad operativa, el modelo de lotes, las campañas y la interpretacion productiva. El codigo RENSPA no sustituye la geometria ni la identidad interna.
-------------------------	--

ACTUALIZACION v4.7 - CARTOGRAFIA DE LOTES, CORRECCIONES Y FUENTES DE INGESTA

E. Construcción posterior de los LoteBase

El perímetro del Establecimiento define la base espacial sobre la que se construyen los lotes internos. Los LoteBase no se inventan ni se crean como placeholders: se incorporan cuando su geometría real es conocida y validable.

- El usuario podrá aportar una **foto, captura, plano o mapa de referencia** donde se identifiquen las divisiones reales del campo; esa referencia sirve para asistir el mapeo de los lotes, pero no se convierte por sí sola en geometría autoritativa.
- El flujo futuro puede permitir digitalización/dibujo sobre mapa y/o importación de archivos geoespaciales soportados. El resultado confirmado se normaliza y persiste en PostGIS.
- Cada LoteBase debe pertenecer a un único Establecimiento y quedar dentro de su límite. Un Establecimiento puede contener tantos lotes internos como requiera la operación.
- La política exacta de solapamiento entre LoteBase se define antes de habilitar el flujo final de dibujo/importación. Se conserva el requisito previo de controlar solapamientos; v4.7 no lo elimina ni lo resuelve prematuramente.
- MapLibre sigue siendo la superficie de visualización prevista: click sobre polígono -> ficha contextual. La captura/edición cartográfica se materializa solamente cuando el slice correspondiente la implemente.

F. Corrección y versionado geográfico

La geometría territorial no se considera eternamente inmutable. Puede existir una corrección legítima de un perímetro SENASA/RENSPA o de un LoteBase interno. Sin embargo, la política de no borrado destructivo y preservación de historia permanece vigente.

- Slice 3.1 no expone una API de edición de geometría ni define todavía el workflow completo de corrección/versionado.
- Una corrección válida nunca puede romper invariantes espaciales existentes; por ejemplo, reducir un Establecimiento no puede dejar un LoteBase actual fuera de su perímetro.
- Cuando Campañas/UOP ya referencien geometría territorial, el flujo soportado de corrección debe preservar la historia necesaria para que una campaña pasada no cambie silenciosamente por editar el presente.
- No se corrigen errores borrando entidades confirmadas. Se utilizan corrección, reemplazo/versionado, anulación o cierre de vigencia según el tipo de dato y la regla que se apruebe en el slice correspondiente.

G. Obtención del perímetro SENASA/RENSPA

La definición de RENSPA/SENASA como fuente inicial **no implica asumir una API pública ni un mecanismo técnico específico**. La ingestión se mantiene separada del dominio y se diseña cuando exista evidencia del mecanismo real disponible para la empresa.

- Puede resolverse mediante archivo/exportación oficial, carga manual controlada, importación cartográfica o un adapter de integración si posteriormente existe un mecanismo oficial y estable.
- No se implementa scraping como dependencia estructural del producto.
- Credenciales, permisos de terceros y detalles de transporte quedan fuera del dominio Territorio y, si aparecen, se encapsulan detrás de un adapter reemplazable.
- La ausencia temporal de ingestión automática no impide operar: un Establecimiento puede cargarse con una geometría validada por un mecanismo soportado y con su procedencia registrada.

REGLA DE CARGA

Una imagen o referencia visual ayuda a digitalizar; el dato operativo final es la geometría PostGIS validada. Un RENSPA ayuda a identificar/proveer contexto; no se interpreta como una lista de vértices.

ACTUALIZACION v4.7 - ETAPA 3 TERRITORIO AMPLIADA

H. Etapa 3 - Territorio, campañas y cartografía

Se conserva íntegramente el alcance de Etapa 3 definido en v4.6. La actualización v4.7 lo precisa con la jerarquía territorial y propone una secuencia de implementación por slices sin alterar el roadmap general.

Slice	Resultado
3.1 - Fundación territorial	Materializar Establecimiento y LoteBase con UUID, códigos canónicos, geometría PostGIS, pertenencia organizacional, referencia SENASA/RENSPA cuando corresponda, procedencia, contención e invariantes. Permiso canónico mínimo: territorio:crear .
3.2 - Campañas y UOP	Materializar Campaña y UOP con historia temporal. Permitir subdividir/agrupar lotes de manera distinta entre campañas sin alterar campañas anteriores. Definir contención y reglas de solapamiento UOP.
3.3 - Actividad/uso territorial	Asignar cultivo/uso a UOP por campaña conforme al modelo conceptual, sin adelantar reglas propias de Agricultura que pertenezcan a Etapa 5.
3.4 - API territorial	Exponer solamente los casos de uso y consultas necesarios para operar/navegar territorio, reutilizando autorización, audit e idempotencia cuando corresponda.
3.5 - Mapa	Mostrar Establecimientos, LoteBase y contexto vigente en MapLibre; click en polígono abre ficha contextual. Mantener separación entre visualización y reglas autoritativas del backend.
3.6 - Captura/importación geográfica	Incorporar flujo soportado para perímetros/lotas: importación geoespacial y/o dibujo/digitalización sobre referencia cartográfica. Las fotos/capturas/planos aportados por el usuario pueden utilizarse como referencia para mapear LoteBase.
3.7 - Cierre Etapa 3	Validar dos campañas con subdivisiones diferentes, superficie/consistencia, E2E del mapa, migraciones limpias, PostGIS real, CI y staging.

I. Pruebas y validaciones territoriales ampliadas

- Geometrias de Establecimiento y LoteBase validas, no vacias, con SRID correcto y persistidas en PostgreSQL/PostGIS real.
- LoteBase contenido/cubierto por su Establecimiento; tocar el borde es valido; quedar parcial o totalmente afuera debe fallar.
- Consistencia de organizacion, padre y autoria protegida tambien ante SQL directo.
- RENSPA/SENSA asociado mediante referencia externa sin reemplazar el UUID interno ni acoplar el dominio a la fuente externa.
- Establecimiento sin LoteBase es un estado valido durante la carga progresiva de cartografia.
- Cuando se materialicen UOP: campaña 1 y campaña 2 pueden subdividir/agrupar los mismos lotes de forma distinta y la campaña 1 permanece identica.
- Política de solapamiento, suma de superficies y tolerancias se prueban con PostGIS y property tests cuando dichas reglas se materialicen.
- E2E final de Etapa 3: mapa -> establecimiento/lote/UOP -> ficha -> campaña/actividad correcta.

J. Criterio de salida ampliado de Etapa 3

CRITERIO DE SALIDA	Ademas de todos los criterios preexistentes, Agro Ops puede representar multiples establecimientos reales de la empresa, cada uno con su perimetro completo y referencia SENASA/RENSPA cuando corresponda, dividir cada establecimiento en multiples LoteBase internos y representar al menos dos campañas con UOP diferentes sin perdida de historia. La cartografia visible permite navegar esas capas y toda geometría confirmada cumple invariantes PostGIS y trazabilidad de origen.
---------------------------	---

ACTUALIZACION v4.7 - IMPACTO EN EL PLAN, DECISIONES Y PRIORIDAD

K. Secciones del Plan v4.6 que quedan ampliadas

- **1 / 2. Resumen y principios:** se explicita que una organizacion puede operar varios establecimientos y que Establecimiento, LoteBase y UOP son capas distintas con ownership e historia propios.
- **3. Arquitectura y ownership:** PostgreSQL/PostGIS conserva la geometria operativa; SENASA/RENSPA se modela como fuente/referencia externa y no contamina el dominio con IDs o protocolos externos.
- **5. CI/CD y pruebas:** se agregan invariantes de geometria de Establecimiento, contencion LoteBase -> Establecimiento, indices espaciales, procedencia y pruebas reales PostGIS.
- **8A. Etapa 1.5 / DER:** el modelo conceptual territorial se precisa como Organizacion -> Establecimiento -> LoteBase -> Campaña/UOP, manteniendo la separacion entre DER conceptual y schema incremental.
- **9. Etapa 2:** se reutilizan ExternalReference, autorizacion, audit e idempotencia. Se incorpora el permiso canonico minimo territorio:crear al materializar la operacion correspondiente; no se reutiliza agricultura:crear como permiso territorial.
- **10. Etapa 3:** se amplian flujo, cartografia, fuente inicial de perimetro, divisiones internas, ingreso posterior desde imagen/plano/mapa y estrategia de correccion geografica.
- **12 / 14 / 19. Agricultura, Ganaderia y Costos:** continúan consumiendo territorio/UOP como contexto; no adquieren ownership sobre los perimetros ni lotes.
- **18. Integracion Finnegans:** si Finnegans requiere dimensiones territoriales se sincronizan codigos/referencias aprobadas; no se transfieren poligonos ni se convierte Finnegans en fuente geografica.
- **21. Piloto/cutover:** el inventario territorial del piloto debe incluir establecimientos, referencias externas, geometrias y lotes internos validados antes de operar verticales dependientes.
- **23. Matriz de pruebas:** Territorio incorpora validacion del perimetro de Establecimiento, LoteBase internos, contencion, historia entre campañas y origen de geometria.
- **24. ADR:** esta precision no cambia los componentes ni la arquitectura base y no requiere ADR por si sola. Una integracion estructural nueva con SENASA o un cambio del modelo de ownership geografico si debiera evaluarse como decision arquitectonica.

L. Decisiones pendientes y POC - actualizacion territorial

Tema	Estado / decision v4.7
Fuente inicial de perimetro de Establecimiento	RESUELTA EN CONCEPTO: utilizar informacion georreferenciada SENASA/RENSPA cuando este disponible y sea confiable para el campo correspondiente; conservar procedencia.
Mecanismo tecnico de obtencion SENASA/RENSPA	PENDIENTE: validar con datos reales de la empresa si llega por exportacion, archivo, carga manual o interfaz oficial. No asumir API publica.
Geometria de LoteBase	RESUELTA EN CONCEPTO: subdivision interna propia de Agro Ops dentro del Establecimiento. Se digitaliza/importa a partir de la informacion real disponible; puede utilizar foto/captura/plano/mapa como referencia.
Solapamiento entre LoteBase	PENDIENTE: definir politica funcional y tolerancias antes de habilitar el flujo final de mapeo. El requisito de control previo se mantiene.
Correccion/versionado geografico	PENDIENTE DE WORKFLOW: permitir correcciones legitimas sin borrado destructivo y preservar historia cuando existan campañas/UOP vinculadas.
Superficie en hectareas	PENDIENTE DE IMPLEMENTACION: derivar desde geometria con estrategia geodesica/proyectada correcta y Decimal/NUMERIC; no duplicar valor editable.

M. Roadmap y regla de compatibilidad

El orden general continua sin cambios: **0 -> 1 -> 1.5 -> 2 -> 3 -> 4 -> ... -> 14**. v4.7 no crea una etapa nueva; profundiza Etapa 3 y reutiliza las primitivas ya construidas en Etapa 2. La secuencia por slices es una disciplina de implementacion y no reemplaza el alcance original del plan.

REGLA DE PRIORIDAD

La actualizacion territorial es estrictamente aditiva. **No elimina, sustituye, posterga ni reduce ningun requerimiento anterior:** autenticacion, autorizacion, Panel Administrativo, Internal Console, audit, jobs, outbox, Documents, idempotencia, Stock, Agricultura, Maquinaria, Ganaderia/XRS2i, Granos, Comercializacion, ARCA, Finnegans, Costos/MB, Conciliacion, Piloto, staging, CI, HTTP/2+, seguridad, no-delete, Decimal y todos los criterios de salida previos mantienen su alcance y prioridad.

ACTUALIZACION v4.8 - EVIDENCIA TERRITORIAL REAL SENASA/RENSPA

CARACTER DE LA ACTUALIZACION

La version 4.8 conserva integramente el Plan Integral v4.7 y todas las actualizaciones anteriores. Esta ampliacion incorpora evidencia obtenida de establecimientos reales de la empresa en el sistema autenticado RENSPA de SENASA y corrige/precisa el modelo de correspondencia entre Establecimiento Agro Ops, RENSPA y geometria fuente. No elimina, sustituye, posterga ni reduce ningun requerimiento, etapa, prueba, integracion, ADR o criterio de salida existente.

A. Hallazgos confirmados con datos reales

Hallazgo	Decision / impacto v4.8
SENASA conserva vertices del perimetro	El flujo autenticado RENSPA expone un campo Poligono editable con una secuencia cerrada de pares (latitud,longitud), ademas de un punto de referencia y un area dibujada. Esto confirma que existe geometria vectorial utilizable sin interpretar el numero RENSPA como coordenadas.
No todo RENSPA tiene poligono disponible	El inventario real incluye al menos un caso con referencia/punto pero sin poligono disponible. La falta de geometria fuente no autoriza a inventar un perimetro ni a derivarlo de un punto o de una superficie declarada.
Un campo de negocio puede abarcar varios RENSPA	La realidad observada en La Manuela muestra varias referencias RENSPA que pueden pertenecer al mismo campo entendido por la empresa. Por lo tanto, RENSPA y Establecimiento Agro Ops no son entidades 1:1.
El nombre SENASA no define la identidad de negocio	Una fraccion puede tener un nombre externo distinto y aun ser candidata a integrar el mismo campo logico. Los nombres de origen se preservan como evidencia, pero no reemplazan el codigo/UUID canonico de Agro Ops.
Geometrias separadas pueden formar un solo campo	La geometria canonica de un Establecimiento puede ser MultiPolygon y conservar componentes desconectados. No se usa convex hull, envelope ni relleno artificial para unir fracciones separadas.

DECISION DE ARQUITECTURA

Establecimiento Agro Ops representa el campo logico reconocido por la empresa. Puede poseer cero, una o varias referencias SENASA/RENSPA. Cada geometria fuente se conserva individualmente con su procedencia; la geometria canonica del Establecimiento se obtiene solamente de fuentes confirmadas y puede ser MultiPolygon. La segmentacion administrativa externa no dicta automaticamente la identidad territorial interna.

ACTUALIZACION v4.8 - IDENTIDAD, FUENTES Y GEOMETRIA CANONICA

B. Distincion entre entidad de negocio y registro externo

- **Establecimiento Agro Ops:** campo logico/operativo de la empresa, con UUID y codigo canonico propios. Sigue siendo la entidad territorial consumida por Campanas, UOP, Agricultura, Ganaderia, Maquinaria y Costos.
- **RENSPA/SENASA:** referencia externa opaca. Varias referencias pueden apuntar al mismo Establecimiento cuando el negocio confirme que representan fracciones del mismo campo.
- **Registro geografico fuente:** evidencia de origen asociada a una referencia externa o a otra fuente soportada. Conserva el nombre de origen, el poligono original/normalizado, punto de referencia, area informada y metadata de captura cuando existan.
- **LoteBase:** subdivision interna propia de Agro Ops. Un RENSPA o fraccion SENASA no se convierte automaticamente en LoteBase.
- **UOP:** subdivision operativa por Campana. No adquiere ownership sobre referencias SENASA ni sobre la geometria fuente.

C. Geometria fuente y geometria canonica

Dato	Tratamiento
Vertices SENASA	Se conserva el valor fuente y se normaliza de (latitud,longitud) a coordenadas geoespaciales [longitud,latitud] con SRID 4326. Se valida cierre, validez y no-vacio antes de confirmarlo.
Poligono por RENSPA	Se preserva por separado para trazabilidad y diagnostico, aun cuando varias referencias terminen agrupadas bajo un mismo Establecimiento.
Geometria canonica	Union espacial exacta de las geometrias fuente confirmadas que componen el Establecimiento, normalizada a geometry(MultiPolygon,4326). No se agregan areas intermedias inexistentes.
Solapamientos entre fuentes	Se detectan y reportan. La union canonica no debe contar dos veces una misma superficie; los originales permanecen disponibles para explicar la diferencia.
Area SENASA	Metadata de referencia de la fuente, util para control. La superficie autoritativa de Agro Ops se deriva de la geometria canonica con estrategia geodesica/proyectada correcta.
Punto lat/lon SENASA	Punto de referencia/ubicacion. Puede conservarse como metadata, pero nunca sustituye el perimetro.

REGLA DE UNION

Cuando varias referencias RENSPA formen un mismo campo, la operacion geometrica canonica debe equivaler a `ST_UnaryUnion(ST_Collect(...))` sobre las fuentes confirmadas y luego normalizarse a `MultiPolygon`. No se utiliza convex hull para hacer que el campo "se vea unido" si existen huecos o fracciones separadas.

ACTUALIZACION v4.8 - INGESTA SENASA Y ESTADOS INCOMPLETOS

D. Mecanismo SENASA confirmado para V1

- Se confirma un mecanismo real y disponible hoy: dentro del flujo autenticado de RENSPA/SENASA, el usuario puede consultar el campo Poligono con vertices (latitud,longitud), junto con punto de referencia y area dibujada.
- V1 no depende de una API publica de SENASA. La via base puede ser carga manual controlada/copia del poligono y, si existe archivo oficial, importacion del archivo.
- No se implementa scraping como dependencia estructural. Si mas adelante se valida una API/interfaz oficial estable, se agrega detras de un adapter sin modificar el dominio territorial.
- El importador debe previsualizar fuente, referencia RENSPA, nombre externo, vertices recibidos, geometria normalizada, validez, area derivada y diferencias antes de confirmar.
- La misma fuente importada nuevamente debe ser idempotente mediante fingerprint/hash y no duplicar referencias ni geometria.

E. Establecimientos sin poligono fuente disponible

La evidencia real muestra que puede existir una referencia RENSPA valida sin poligono utilizable en la pantalla disponible. Este caso no debilita la invariante de Establecimiento.geometria no nula definida en v4.7. En su lugar se distingue entre inventario/candidato de ingesta y Establecimiento territorial confirmado.

Estado	Regla
Fuente completa	RENSPA + poligono validable: puede participar en la confirmacion o actualizacion controlada del Establecimiento.
Fuente incompleta	RENSPA/punto/superficie pero sin poligono: se conserva como pendiente de ingesta o evidencia, pero no se fabrica geometria canonica.
Resolucion	Obtener archivo/fuente confiable, digitalizar manualmente mediante flujo soportado o incorporar otra fuente cartografica. Solo despues se confirma la geometria.

CASO PILOTO

Los Ceibos queda como caso obligatorio de geometria pendiente: la aplicacion debe poder registrar el pendiente de ingesta sin inventar un perimetro y conducir al usuario hacia una resolucion soportada.

F. Dataset territorial piloto versionado

Los datos reales utilizados para validar Etapa 3 deben conservarse en el repositorio como dataset/fixture documental versionado, separado de migrations y de seeds de produccion. Debe incluir los registros individuales, las geometria fuente normalizadas, los RENSPA, estados pendientes y las agrupaciones propuestas. Los datos de origen nunca se reemplazan por la geometria agrupada.

ACTUALIZACION v4.8 - IMPACTO EN LOS SLICES DE ETAPA 3

G. Secuencia de Etapa 3 ajustada sin eliminar alcance

Slice	Ajuste v4.8
3.1 - Fundacion territorial	CERRADO, sin rollback. Se mantiene Establecimiento/LoteBase y geometry(MultiPolygon,4326). Se explicita que un Establecimiento puede tener multiples ExternalReference SENASA/RENSPA y que el modelo no debe asumir un campo rensa singular.
3.2 - Campanas y UOP	CERRADO, sin cambios. Las Campanas/UOP dependen del Establecimiento canonico, no de la particion administrativa RENSPA.
3.3 - Actividad/uso territorial	CERRADO, sin cambios. El uso territorial sigue asociado a UOP por vigencia y no conoce fuentes SENASA.
3.4 - API territorial	Debe exponer Establecimiento canonico y coleccion de referencias externas; evitar contratos que presupongan rensa: string unico. La geometria operativa se entrega como GeoJSON/MultiPolygon. No implementar aun importacion/edicion geografica.
3.5 - Mapa	MapLibre representa el Establecimiento canonico. Si es MultiPolygon, cualquier componente pertenece a la misma ficha. Las fuentes RENSPA individuales no se confunden con LoteBase; una capa tecnica de procedencia puede agregarse cuando 3.6 materialice esos datos.
3.6 - Captura/importacion geografica	Se amplia para importar vertices SENASA, conservar geometrias fuente por referencia, permitir agrupacion many-to-one hacia Establecimiento, calcular union canonica, detectar solapamientos/diferencias, manejar fuentes incompletas y mantener idempotencia/proveniencia.
3.7 - Cierre Etapa 3	Debe validar un campo compuesto por multiples RENSPA, un MultiPolygon con componentes desconectados, conservacion de fuentes individuales, un caso sin geometria fuente, dos Campanas con UOP distintas, areas, mapa, API, migrations, CI y staging.

H. Refinamiento interno recomendado para Slice 3.6

- **3.6a - Parser/normalizacion SENASA:** texto (latitud,longitud) -> geometria 4326, validaciones y preview.
- **3.6b - Fuentes y procedencia:** conservar original + normalizado + ExternalReference + fingerprint/import metadata.
- **3.6c - Agrupacion a Establecimiento:** asociar una o varias fuentes confirmadas a un campo logico y recomputar union exacta.
- **3.6d - Dibujo/import alternativo:** resolver establecimientos sin poligono SENASA y carga de LoteBase desde archivos/mapa/referencia visual.
- **3.6e - Correccion/versionado:** corregir fuente o agrupacion sin borrado destructivo y sin alterar silenciosamente historia de Campanas/UOP.

DISCIPLINA	Estos sub-bloques refinan Slice 3.6 para mantener una tarea coherente por implementacion. No crean una nueva etapa ni eliminan la captura/importacion geografica ya prevista en v4.7.
-------------------	---

ACTUALIZACION v4.8 - PRUEBAS, PILOTO Y DECISIONES

I. Pruebas territoriales adicionales

- Un Establecimiento puede tener varias ExternalReference SENASA/RENSPA sin duplicar la entidad de negocio.
- Dos RENSPA distintos pueden conservar sus geometrias fuente separadas y, tras confirmacion de negocio, contribuir a una unica geometria canonica.
- La union de componentes separados produce MultiPolygon y conserva huecos/distancias reales; ningun test acepta convex hull como sustituto.
- La union de fuentes solapadas no duplica superficie y conserva evidencia para explicar el solapamiento.
- Parser SENASA valida anillo cerrado, orden de coordenadas, SRID, geometria valida/no vacia y normaliza Polygon/MultiPolygon.
- Area fuente SENASA puede diferir de area derivada; la diferencia genera control/advertencia, no reemplazo de la geometria.
- Punto de referencia SENASA no puede utilizarse como perimetro.
- Reimportar la misma fuente no duplica geometria, ExternalReference ni efectos.
- Una fuente sin poligono queda pendiente y no genera un Establecimiento canonico con geometria inventada.
- Correccion/reagrupacion futura preserva la historia territorial requerida por Campanas/UOP.

J. Caso piloto La Manuela

El dataset real contiene varias referencias con nombre La Manuela y una fraccion denominada Martina Palazon espacialmente contigua a La Manuela 27-29. La arquitectura considera valida la hipotesis de agruparlas en un unico Establecimiento logico si el negocio la confirma. La confirmacion empresarial de Martina Palazon como parte de La Manuela es una decision de datos del piloto, no una condicion para 3.4 ni una razon para cambiar la arquitectura.

REGLA DEL PILOTO

Guardar siempre ambas vistas: (1) cada registro RENSPA/geometria fuente por separado, con su nombre externo; y (2) la agrupacion canonica confirmada por la empresa. La agrupacion nunca destruye ni reescribe la evidencia de origen.

K. Decisiones pendientes y POC - estado v4.8

Tema	Estado / decision v4.8
Fuente SENASA de poligono	RESUELTA PARA V1 MANUAL CONTROLADA: el portal autenticado RENSPA expone vertices del poligono. No se asume API publica.
Automatizacion SENASA	PENDIENTE / OPCIONAL: investigar interfaz oficial estable. Si existe, adapter reemplazable. Sin scraping estructural.
RENSPA -> Establecimiento	RESUELTA EN CONCEPTO: many-to-one permitido. La empresa define el campo logico; RENSPA queda como referencia/fuente externa.
Geometria canonica agrupada	RESUELTA EN CONCEPTO: union exacta de fuentes confirmadas, normalizada a MultiPolygon; preservar originales.
Martina Palazon / La Manuela	PENDIENTE DE CONFIRMACION DE NEGOCIO: espacialmente compatible; confirmar inclusion antes de cargar agrupacion definitiva.
Los Ceibos	PENDIENTE DE GEOMETRIA: conservar referencia/punto como evidencia y resolver por fuente alternativa o dibujo/import soportado.
Solapamiento LoteBase	SIGUE PENDIENTE: v4.8 no altera la decision funcional/tolerancias requeridas por v4.7.

REGLA DE PRIORIDAD

v4.8 es estrictamente aditiva. No elimina, sustituye, posterga ni reduce autenticacion, autorizacion, Panel Administrativo, Internal Console, audit, jobs, outbox, Documents, idempotencia, Stock, Agricultura, Maquinaria, Ganaderia/XRS2i, Granos, Comercializacion, ARCA, Finnegans, Costos/MB, Conciliacion, Piloto, staging, CI, HTTP/2+, seguridad, no-delete, Decimal, SOLID, alta cohesion, bajo acoplamiento, modularidad, abstraccion ni ningun criterio de salida anterior.

ACTUALIZACION v4.9 - EXPERIENCIA DE PRODUCTO, UX/UI Y SISTEMA DE DISEÑO

CARACTER DE LA ACTUALIZACION

La version 4.9 conserva integramente el Plan Integral v4.8 y todas las actualizaciones anteriores. Esta ampliacion agrega una disciplina transversal de experiencia de usuario, interfaz y sistema de diseno para que los flujos ya exigidos no solo existan y sean correctos, sino que formen un producto coherente, claro, eficiente, accesible y visualmente cuidado. No elimina, sustituye, posterga ni reduce ningun requisito funcional, regla de dominio, prueba, integracion, ADR, criterio de seguridad ni criterio de salida previo.

La nueva disciplina no convierte el frontend en autoridad de negocio. Rust/PostgreSQL/PostGIS y las reglas ya definidas conservan sus responsabilidades. UX/UI gobierna como se presentan y operan esos flujos, sin duplicar validaciones autoritativas ni introducir reglas funcionales paralelas.

DECISION DE PRODUCTO

Agro Ops debe sentirse como un unico producto operativo. Todo flujo visible relevante debe cumplir simultaneamente correccion funcional, coherencia visual, eficiencia operativa y accesibilidad. El diseno no se deja para una reescritura final: se establece una base antes de Etapa 4, se aplica en cada slice con interfaz y se revisa nuevamente con producto real durante el roadmap.

A. Que agrega v4.9

Area	Decision / Impacto v4.9
Gobierno UX/UI	Se incorpora una fuente de verdad versionada de diseno y dos skills repo-locales especificas para implementar y revisar UX/UI.
Bootstrap inicial	Se reconoce explicitamente que hoy las skills UX/UI externas no estan instaladas y que DESIGN.md no existe. El primer trabajo UX debe preparar esas herramientas antes de usarlas.
Roadmap	Se inserta UX-0 despues de 3.6e y antes de 3.7. No altera la numeracion 0 -> 1 -> 1.5 -> 2 -> 3 -> ... -> 14.
Slices futuros	Todo slice con interfaz nueva o modificacion visible relevante debe aplicar el contrato UX/UI aprobado; los slices puramente backend/DB no cargan skills de diseno innecesariamente.
Calidad	Definition of Done y matriz de pruebas incorporan accesibilidad, responsive, estados UX, consistencia visual y E2E de los flujos visibles sin reemplazar tests funcionales existentes.
Evolucion	Se agrega un checkpoint intermedio UX-1 despues de Etapa 7 y un hardening UX final dentro de Etapa 14.

ACTUALIZACION v4.9 - PRINCIPIOS UX/UI NO NEGOCIABLES

B. Principios de experiencia de producto

Principio	Regla
Operacion antes que decoracion	La estetica debe mejorar lectura, decision y velocidad de uso. No se agregan ornamentos que oculten datos, reduzcan contraste o aumenten pasos.
Un lenguaje visual	Navegacion, formularios, tablas, fichas, mapas, estados, mensajes y acciones comparten patrones reconocibles en todos los dominios.
Español funcional del agro	Copy, etiquetas, errores y ayudas usan vocabulario natural para una PyME agricola-ganadera argentina. Convenciones tecnicas permanecen internas cuando corresponda.
Jerarquia y densidad intencional	La interfaz prioriza informacion accionable y admite densidad alta donde la operacion lo requiere, sin convertirse en una grilla ilegible.
Prevencion de errores	Acciones destructivas o irreversibles se evitan segun las reglas existentes; la UI explica precondiciones, bloqueos y consecuencias antes de confirmar.
Estados completos	Todo flujo relevante contempla loading, vacio, exito, error, permiso insuficiente, dato incompleto y reintento cuando aplique.
Responsive real	Desktop es una superficie operativa principal, pero tablet y celular deben conservar navegacion y tareas soportadas; no se acepta solamente "achicar" desktop.
Accesibilidad	Objetivo general: WCAG 2.2 AA en superficies de usuario razonablemente aplicables, incluyendo teclado, focus visible, contraste, labels y semantica.
Mapas operativos	MapLibre se trata como superficie de trabajo: seleccion, contexto, capas, dibujo y validacion deben ser comprensibles y no depender solo de color.
Rendimiento percibido	Los flujos deben responder con feedback inmediato, evitar bloqueos visuales innecesarios y no introducir dependencias pesadas sin beneficio medible.

C. Fuente de verdad de diseno

DESIGN.md sera la fuente de verdad versionada para decisiones de experiencia y presentacion de Agro Ops una vez creado en UX-0. No existe al inicio de v4.9 y por lo tanto ningun prompt anterior debe asumirlo.

REGLA DE AUTORIDAD

Las skills externas aportan heurísticas y conocimiento; **no gobiernan la identidad del producto**. Despues de UX-0, DESIGN.md y las skills repo-locales de Agro Ops tienen prioridad. Una actualizacion de una skill upstream no puede cambiar silenciosamente el aspecto o comportamiento del producto.

ACTUALIZACION v4.9 - UX-0a

BOOTSTRAP DE SKILLS Y GOBIERNO

D. Primer step obligatorio: preparar el entorno UX/UI

UX-0a existe para evitar una dependencia circular: hoy no estan instaladas las skills UX/UI propuestas y tampoco existe DESIGN.md. Por lo tanto, el primer slice de esta disciplina **no debe intentar rediseñar la aplicacion**. Debe dejar listo el mecanismo de trabajo reproducible.

Secuencia UX-0a

Paso	Accion	Criterio de salida
UX-0a.1 - Inventario	Confirmar skills ya presentes bajo .agents/skills, workflow vigente, stack frontend y superficies visibles actuales. No hacer analisis general del repo fuera de lo necesario.	Inventario corto y sin asumir herramientas inexistentes.
UX-0a.2 - Seleccion upstream	Verificar en ese momento repositorio, licencia, compatibilidad e instrucciones de las candidates frontend-design, web-design-guidelines y react-best-practices. UI UX Pro Max queda opcional para exploracion, no como autoridad automatica.	Lista aprobada/rechazada con origen y revision fijados.
UX-0a.3 - Instalacion/pinning	Instalar o incorporar las skills aprobadas de forma compatible con Codex. Registrar procedencia, version/commit y fecha para evitar cambios silenciosos.	Codex puede resolverlas de forma reproducible.
UX-0a.4 - Skills Agro Ops	Crear .agents/skills/agro-ops-ui-design/SKILL.md y .agents/skills/agro-ops-ui-review/SKILL.md. Deben conocer el producto, respetar AGENTS.md y usar DESIGN.md cuando exista.	Ambas skills funcionan localmente y no dependen de recordar prompts largos.
UX-0a.5 - Workflow	Actualizar Agro Ops Codex Workflow & Skills Context con matriz de uso, modelo recomendado y regla de no cargar skills UI en slices que no tienen interfaz.	Workflow versionado y coherente con 1 slice = 1 tarea.
UX-0a.6 - Smoke	Ejecutar una prueba pequena de descubrimiento/invocacion de las skills sin modificar pantallas de produccion.	Skills resolubles; ningun rediseño accidental.

SIN BLOQUEO POR TERCEROS

Si una skill upstream deja de estar disponible o no puede instalarse de forma confiable, UX-0a debe conservar el contrato necesario en las skills repo-locales. La calidad UX/UI no puede depender de disponibilidad en vivo de un repositorio externo.

Modelo recomendado para UX-0a

Sol High para seleccionar el esquema de gobierno y resolver incompatibilidades reales. Una vez decidido el diseño de skills, **Terra Medium/High** puede hacer la instalacion, archivos repo-locales y ajustes mecanicos de workflow. No se mezcla en este slice la refactorizacion visual de la aplicacion.

ACTUALIZACION v4.9 - UX-0b CONTRATO DE DISEÑO

E. Crear DESIGN.md antes de aplicar el sistema visual

Con UX-0a cerrado, UX-0b estudia las superficies ya construidas y fija el contrato de producto. El objetivo no es generar una colección de estilos arbitrarios, sino decidir como debe sentirse Agro Ops y convertir esas decisiones en reglas ejecutables y revisables.

Contenido mínimo de DESIGN.md

Bloque	Debe definir
Principios de producto	Claridad, velocidad operativa, densidad, prevención de errores, lenguaje funcional, comportamiento consistente.
Arquitectura de navegación	Shell, jerarquía principal, sidebar/header si corresponde, breadcrumbs/contexto, patrones de cambio entre dominios y retorno a contexto.
Sistema visual	Tipografía, escala, espaciado, radios, bordes, superficies, elevación, iconografía y uso semántico del color. La paleta final debe ser una decisión deliberada, no una default de la skill.
Componentes y patrones	Botones, inputs, selects, dialogs/drawers, tablas, filtros, fichas, badges, alertas, tabs, empty states, skeletons y confirmaciones.
Formularios operativos	Orden de campos, validación progresiva, errores cercanos al campo, keyboard flow, defaults seguros, confirmación y recuperación.
Mapas	Jerarquía mapa/panel, selección, hover/focus, leyendas, capas, dibujo, importación, errores de geometría, feedback de guardado y uso sin depender exclusivamente de color.
Responsive	Breakpoints/patrones por capacidad, no solo por ancho; que tareas se conservan o se simplifican en tablet/celular.
Accesibilidad	Focus, teclado, semántica, contraste, labels, mensajes, áreas táctiles y criterios WCAG aplicables.
Motion y feedback	Animaciones discretas con propósito, respeto por reduced motion, feedback inmediato y estados de transición.
Contenido	Terminología, formato de fechas/numeros/unidades, microcopy, tono de errores y ayudas en español del agro argentino.
Gobierno	Como se aprueba una nueva variante/componente, cuando modificar DESIGN.md y cuando una decisión requiere ADR por impacto arquitectónico.

APROBACION DE PRODUCTO

UX-0b debe producir una propuesta concreta y revisable. Antes de convertirla en refactor amplio, el responsable del producto valida la dirección visual y los patrones principales. Codex no tiene autoridad para redefinir por sí solo la identidad visual de Agro Ops.

ACTUALIZACION v4.9 - UX-0c FUNDACION IMPLEMENTADA

F. Materializar la base antes de cerrar Etapa 3

UX-0c implementa la direccion aprobada de UX-0b sobre la arquitectura web existente. No reemplaza Next.js/React/MapLibre ni introduce un framework visual nuevo por conveniencia. Se reutilizan las abstracciones actuales y se agregan solamente tokens/componentes necesarios para sostener consistencia.

Alcance minimo

Area	Resultado esperado
Design tokens	Variables/abstracciones para color semantico, tipografia, espaciado, superficies y estados. Sin duplicar valores arbitrarios en cada modulo.
App shell	Navegacion y estructura base coherentes para superficies autenticadas, respetando permisos y rutas existentes.
Primitives utiles	Solo componentes que ya tienen consumidores reales o inmediatos. No crear una libreria especulativa de decenas de componentes.
Estados comunes	Loading, empty, error, success, disabled y permission denied consistentes donde correspondan.
Territorio como referencia	Alinear /territorio con el sistema: mapa dominante cuando el trabajo sea geografico, panel/ficha contextual, acciones claras de dibujo/importacion y feedback de validacion.
Configuracion y dashboard	Corregir inconsistencias evidentes para que las superficies ya existentes no entren a Etapa 4 con un lenguaje visual distinto.
Responsive/accesibilidad	Aplicar la base a los flujos que se toquen; no declarar cobertura total sin pruebas reales.

Fuera de alcance de UX-0c

- No cambia reglas de dominio, autorizacion, idempotencia, transacciones, fuentes de verdad ni contratos API salvo que un defecto real lo exija y se trate como slice separado.
- No reescribe todo el frontend ni crea una segunda arquitectura de componentes.
- No intenta completar UI de etapas futuras ni inventa features para llenar pantallas.
- No reemplaza E2E, tests de backend o validaciones PostGIS por pruebas visuales.

CRITERIO DE CIERRE UX-0

Skills instaladas y gobernadas + skills repo-locales + DESIGN.md aprobado + base visual implementada + superficies actuales relevantes alineadas + verificacion frontend/UX proporcional. Recién entonces 3.7 puede validar Etapa 3 tambien como experiencia de producto.

ACTUALIZACION v4.9 - USO DE SKILLS Y MODELOS CODEX

G. Matriz de invocacion

Tipo de tarea	Modelo orientativo	Skills	Regla
Arquitectura UX, navegacion, direccion visual, cambio de sistema de disenio	Sol High	\$agro-ops-ui-design + guidelines aprobadas	Resolver decisiones antes de implementar. Si cambia arquitectura tecnica, evaluar ADR.
Implementar flujo visible ya diseñado	Terra High	\$agro-ops-slice + \$agro-ops-ui-design + react-best-practices	Un slice coherente; mantener contrato DESIGN.md y reglas de dominio.
Cambio UI pequeno/mecanico	Terra Medium	\$agro-ops-slice; UI skill solo si afecta patron visual	No cargar contexto UX innecesario.
Review de pantalla/flujo importante	Terra High o Sol High segun riesgo	\$agro-ops-ui-review + web-design-guidelines	Revisar diff/superficie, no reanalizar todo el repo.
Backend/DB sin UI	Segun workflow existente	\$agro-ops-slice / review / verify	No usar skills UI.
Verificacion final	Terra Medium/High	\$agro-ops-verify + checks UX solo si hubo UI	Una pasada proporcional al finalizar, no despues de cada pequeno cambio.

H. Responsabilidades de las skills repo-locales

Skill	Responsabilidad
\$agro-ops-ui-design	Traducir un flujo funcional ya definido a experiencia coherente con DESIGN.md; jerarquizar informacion; reutilizar patrones; cubrir estados; respetar responsive/accesibilidad; no inventar reglas de negocio.
\$agro-ops-ui-review	Auditar una superficie implementada contra DESIGN.md y criterios de usabilidad, accesibilidad, responsive, consistencia, feedback, eficiencia y deuda visual; entregar hallazgos priorizados.

BOOTSTRAP MODE

\$agro-ops-ui-design debe contemplar explicitamente que DESIGN.md puede no existir durante UX-0b. En ese unico contexto puede ayudar a crearlo. Despues de su aprobacion, la ausencia de DESIGN.md es un error de proceso y la skill no debe improvisar una identidad paralela.

ACTUALIZACION v4.9 - ROADMAP Y PUNTOS DE APLICACION

I. Insercion sin renumerar etapas

La secuencia general se conserva. UX-0 es un checkpoint transversal dentro del cierre de Etapa 3, no una nueva etapa funcional numerada.

SECUENCIA	3.6d -> 3.6e -> UX-0a Skills/Gobierno -> UX-0b DESIGN.md -> UX-0c Fundacion UI -> 3.7 Cierre Etapa 3 -> Etapa 4 -> ... -> Etapa 14
------------------	---

J. Aplicacion por etapa

Momento	Aplicacion UX/UI
Etapa 3 actual	3.6d y 3.6e mantienen su alcance territorial. No se los transforma en un rediseño global. UX-0 comienza despues de 3.6e.
Etapas 4-7	Cada slice con pantalla/formulario/mapa/dashboard nuevo aplica DESIGN.md durante la implementacion. El cierre de cada etapa revisa sus flujos visibles principales.
UX-1 - despues de Etapa 7	Auditoria transversal con Stock, Agricultura, Maquinaria y Ganaderia reales. Detectar friccion, divergencia de patrones, densidad incorrecta y deuda responsive. Ajustar DESIGN.md de forma deliberada, no reescribir por moda.
Etapas 8-13	Continuar el contrato consolidado en Granos, Comercializacion, ARCA, Finnegans, Costos y Reportes. Pantallas complejas de conciliacion/tablas reciben review UX especifico.
Etapa 14	El piloto incluye hardening UX final con usuarios y datos representativos: pasos, errores, tiempos percibidos, tablet/celular, accesibilidad, consistencia y polish. Se corrige friccion real antes de cutover.

K. Regla por slice

La presencia de frontend no obliga automaticamente a una review UX separada. Se usa \$agro-ops-ui-review cuando el cambio introduce una pantalla importante, mapa, tabla compleja, formulario complejo, wizard, navegacion nueva, dashboard, cambio responsive significativo o cuando cierra una etapa. Cambios pequenos siguen la disciplina normal y se validan dentro del slice.

ACTUALIZACION v4.9 - DEFINITION OF DONE Y PRUEBAS UX

L. Extension aditiva del Definition of Done

Para un slice con UI relevante	Criterio adicional
Flujo	La tarea completa puede ejecutarse desde la interfaz sin recurrir a UUID, SQL, Railway, CLI ni conocimiento tecnico salvo herramientas expresamente tecnicas del producto.
Consistencia	Usa componentes/patrones existentes o actualiza DESIGN.md de manera deliberada antes de introducir una variante nueva.
Estados	Loading, vacio, error, permiso, validacion y exito estan cubiertos segun aplique; no existe una pantalla que solo funcione en happy path.
Responsive	Los viewports soportados mantienen lectura, acciones y contexto; se prueba al menos desktop y el viewport movil/tablet relevante para el flujo.
Accesibilidad	Navegacion por teclado, focus, labels, semantica, contraste y mensajes cumplen el objetivo definido salvo excepcion documentada.
Performance UX	No se introducen bloqueos, renders o dependencias innecesarias que degraden de forma visible el flujo.
Copy	Textos funcionales en español del agro y errores accionables, sin terminologia interna expuesta al usuario.
Tests	Tests de comportamiento siguen siendo autoritativos. Se agregan checks UI/E2E/a11y donde aportan cobertura real; no se sustituyen invariantes de backend por snapshots.

M. Estrategia de pruebas y verificacion

- **Unit/component:** estados de componentes y contratos de interaccion relevantes.
- **E2E:** flujos visibles criticos ya previstos por etapa, ahora tambien verifican que el usuario puede completar la tarea mediante la UI.
- **Accesibilidad automatizada:** incorporar checks compatibles con el stack en superficies criticas; complementarlos con revision manual de teclado/focus cuando amerite.
- **Responsive:** probar viewports representativos, especialmente en mapas, tablas, formularios y workflows de campo.
- **Regresion visual:** usarla de forma selectiva en superficies estables donde el costo/ruido sea razonable; nunca como unica prueba de correccion.
- **Verificacion:** lint + TypeScript + tests relevantes + build/E2E cuando corresponda, una vez al cierre coherente del slice segun la disciplina ya vigente.

NO CAMBIA

Todas las pruebas existentes de Rust, PostgreSQL/PostGIS, Docker, integraciones, idempotencia, seguridad, auditoria y CI permanecen. v4.9 agrega evidencia de calidad de experiencia; no rebaja ningun gate anterior.

ACTUALIZACION v4.9 - IMPACTO TRANSVERSAL Y PRIORIDAD

N. Secciones del Plan Integral afectadas

Seccion previa	Ajuste v4.9
2 - Principios no negociables	Se agregan coherencia UX/UI, accesibilidad, responsive, estados completos y gobierno de DESIGN.md sin alterar principios de ownership, no-delete, Decimal, GitHub o tests.
3 - Arquitectura	Next.js/React conserva responsabilidad de interfaz. DESIGN.md y skills son artefactos de ingenieria/producto, no nuevos servicios runtime ni nuevas fuentes de verdad.
5 - GitHub/CI/pruebas	Los cambios visibles agregan checks frontend/UX proporcionales. No se impone una suite visual pesada en cada push si no aporta señal.
6 - Estrategia incremental/DoD	La correccion funcional sigue siendo obligatoria y ahora un flujo visible relevante debe cumplir ademas el contrato UX/UI.
10 - Etapa 3	Se inserta UX-0 despues de 3.6e y antes de 3.7; 3.6d/3.6e no pierden ni amplian su responsabilidad territorial.
11-20 - Etapas 4-13	Cada nueva superficie de usuario hereda DESIGN.md y usa skills UI solo donde corresponda.
21 - Etapa 14	El piloto/cutover incorpora hardening de experiencia con uso real y correccion de friccion antes de salida.
23 - Matriz de pruebas	Se agregan a11y, responsive, estados UX y E2E visible como categorias complementarias.
24 - ADR	Cambiar color, spacing o componente no exige ADR. Adoptar un nuevo framework/UI platform, alterar arquitectura frontend o introducir dependencia estructural si debe evaluarse como decision arquitectonica.
25 - Decisiones pendientes	UX-0b debe resolver direccion visual, densidad, navegacion y patrones principales. No se fijan prematuramente en v4.9 valores esteticos concretos.
26 - Roadmap	Se registran UX-0, UX-1 y hardening UX de Etapa 14 como checkpoints transversales sin reenumerar etapas funcionales.

O. Decisiones pendientes especificas de UX-0

Tema	Estado v4.9
Skills upstream exactas/versiones	PENDIENTE DE UX-0a: verificar disponibilidad, licencia, compatibilidad y pinning al momento de instalar.
Direccion visual final	PENDIENTE DE UX-0b: debe surgir de propuesta y aprobacion explicita, no de defaults de una skill.
Implementacion de tokens/componentes	PENDIENTE DE UX-0c: reutilizar la arquitectura actual; nueva libreria solo con justificacion.
Solapamiento LoteBase	SIGUE PENDIENTE: v4.9 no modifica la decision funcional/tolerancias territoriales de v4.7/v4.8.

REGLA DE PRIORIDAD

v4.9 es estrictamente aditiva. No elimina, sustituye, posterga ni reduce autenticacion, autorizacion, Panel Administrativo, Internal Console, audit, jobs, outbox, Documents, idempotencia, Territorio/SENASA/RENSPA, Stock, Agricultura, Maquinaria, Ganaderia/XRS2i, Granos, Comercializacion, ARCA, Finnegans, Costos/MB, Conciliacion, Piloto, staging, CI, HTTP/2+, seguridad, no-delete, Decimal, SOLID, alta cohesion, bajo acoplamiento, modularidad, abstraccion ni ningun criterio de salida anterior.

ACTUALIZACION v4.10 - VISTAS TERRITORIALES FOCALIZADAS Y AGRUPACION OPERATIVA

CARACTER DE LA ACTUALIZACION

La version 4.10 conserva integramente el Plan Integral v4.9 y todas las actualizaciones anteriores. Esta ampliacion precisa como se agrupan las fuentes geograficas reales del piloto en Establecimientos logicos y como debe operar la experiencia cartografica para evitar una navegacion global innecesaria. No elimina, sustituye, posterga ni reduce ningun requerimiento funcional, regla de dominio, prueba, integracion, criterio de seguridad, disciplina UX/UI ni criterio de salida previo.

DECISION DE PRODUCTO

Agro Ops debe presentar el territorio de la empresa como areas de trabajo focalizadas por Establecimiento. En el piloto actual existen cinco Establecimientos logicos de negocio: La Manuela, El Triunfo, El Desafio, San Ignacio y El Rocio. Estos cinco nombres son datos del piloto, no una restriccion estructural: el producto debe continuar soportando N Establecimientos.

A. Cinco areas logicas del piloto

Area logica	Regla v4.10
La Manuela	Establecimiento logico del piloto. Puede agrupar varias fuentes SENASA/RENSPA u otras fuentes confirmadas. El nombre externo de cada fuente se conserva.
El Triunfo	Establecimiento logico del piloto con sus fuentes y poligonos confirmados por posicion espacial y negocio.
El Desafio	Establecimiento logico del piloto. La identidad interna no depende del nombre de una referencia externa.
San Ignacio	Establecimiento logico del piloto. Ceibo se considera parte de San Ignacio por decision de negocio; otras fuentes candidatas se incorporan solo despues de validar ubicacion y confirmacion.
El Rocio	Establecimiento logico del piloto con la misma politica de fuentes, geometria canonica e historia que el resto.

La lista anterior no se codifica en enums, constraints, branches, rutas especiales ni logica Rust. Se materializa mediante datos normales de Establecimiento y relaciones de procedencia. Una futura alta o baja de campos no requiere cambiar codigo por el mero hecho de modificar la cantidad de Establecimientos.

NO HARDCODE

Los cinco campos son la configuracion real de la empresa hoy. La arquitectura sigue siendo generica para multiples Establecimientos y multiples fuentes por Establecimiento.

ACTUALIZACION v4.10 - REGLA DE AGRUPACION POR GEOMETRIA Y NEGOCIO

B. Como se decide que poligonos pertenecen al mismo Establecimiento

La agrupacion no se resuelve por coincidencia de nombre. SENASA/RENSPA conserva nomenclaturas administrativas o historicas que pueden no coincidir con el campo logico utilizado por la empresa. La ubicacion espacial es evidencia necesaria, pero tampoco autoriza por si sola una fusion automatica.

Paso	Regla
1. Evidencia fuente	Conservar cada ExternalReference, nombre externo, geometria, metadata y procedencia por separado.
2. Analisis espacial	Comparar coordenadas, proximidad, continuidad, superposicion y posicion relativa mediante PostGIS. Las fuentes desconectadas pueden integrar un mismo Establecimiento si el negocio lo confirma.
3. Candidato	El sistema puede mostrar una agrupacion candidata o advertir ambigüedades. No confirma silenciosamente por distancia ni por texto.
4. Confirmacion de negocio	Un usuario autorizado confirma a que Establecimiento contribuye la fuente. La decision queda auditada y versionada cuando cambie.
5. Geometria canonica	La geometria vigente del Establecimiento se deriva por union exacta de las contribuciones activas confirmadas y se normaliza a MultiPolygon 4326, sin convex hull ni rellenos artificiales.

C. Casos del piloto ya conocidos

- Ceibo -> San Ignacio: se registra como decision de negocio del piloto. La fuente mantiene su nombre externo Ceibo; no se renombra para simular identidad.
- Fuente denominada "La Manuela" candidata a San Ignacio: permanece PENDIENTE hasta revisar las coordenadas del registro concreto y confirmar el criterio empresarial. v4.10 no la asigna por intuicion ni por similitud de nombre.
- Resto de fuentes: se clasifican con la misma secuencia evidencia espacial -> candidato -> confirmacion. La agrupacion canonica nunca destruye los poligonos originales.

REGLA DE IDENTIDAD	El Establecimiento es la entidad de negocio. El nombre de la fuente externa es evidencia. Una fuente puede llamarse distinto del Establecimiento al que pertenece, y dos fuentes con nombres parecidos pueden pertenecer a campos distintos.
--------------------	--

No se introduce un algoritmo automatico obligatorio de clustering. PostGIS ayuda a medir y visualizar relaciones espaciales; la decision final de pertenencia territorial sigue siendo una decision de negocio auditable cuando exista ambigüedad.

ACTUALIZACION v4.10 - IMPACTO EN SLICE 3.6e

D. Correccion, versionado y reagrupacion geografica

Slice 3.6e conserva su objetivo original de corregir geometrias o agrupaciones sin borrado destructivo y queda precisado para soportar los cinco campos logicos del piloto y cualquier cantidad futura de Establecimientos.

Caso	Comportamiento requerido
Corregir una fuente	Una fuente confirmada no se reescribe para ocultar el valor anterior. La correccion crea una nueva revision/reemplazo trazable y conserva la evidencia previa.
Mover fuente A -> B	Cerrar/versionar la contribucion vigente al Establecimiento A y crear la contribucion vigente al Establecimiento B dentro de una operacion coherente y auditada.
Recomputar canonicas	Recalcular solamente los Establecimientos afectados mediante union exacta de sus contribuciones vigentes. El resultado sigue siendo MultiPolygon 4326 valido y no vacio cuando corresponda.
Historia	Conservar versiones territoriales suficientes para que Campanas/UOP historicas no cambien de significado por una correccion presente. Reutilizar el mecanismo historico existente o agregar la referencia minima necesaria.
Idempotencia	Reintentar la misma correccion/reagrupacion no duplica revisiones, contribuciones, audit ni efectos canonicos.
Autorizacion	La correccion requiere una capacidad territorial de edicion/gestion coherente con la taxonomia existente; la UI nunca sustituye la barrera Rust.

TRANSACCION	Cuando una correccion cambia la pertenencia de una fuente, la historia de la relacion, las nuevas contribuciones, las geometrias canonicas afectadas, idempotencia y audit deben quedar en un estado consistente. No debe existir un intervalo visible donde la fuente contribuya simultaneamente a dos campos por un fallo parcial no intencional.
-------------	---

E. Limites de 3.6e

- 3.6e no implementa todavia el rediseño visual completo de las cinco vistas; eso pertenece a UX-0c.
- 3.6e no inventa agrupaciones de datos del piloto que no hayan sido confirmadas.
- 3.6e no resuelve por accidente la politica pendiente de solapamiento entre LoteBase.
- 3.6e no elimina fuentes viejas, no repara geometria de forma silenciosa y no usa convex hull para simplificar un campo.

ACTUALIZACION v4.10 - MAPAS FOCALIZADOS Y EFICIENCIA

F. Modelo de navegacion cartografica

La experiencia territorial no debe comportarse como un mapa mundial de navegacion libre. El usuario trabaja sobre un Establecimiento seleccionado y puede explorar su perimetro y un entorno razonable, con zoom y paneo locales.

Aspecto	Regla v4.10
Mapa activo	Montar un solo mapa interactivo MapLibre para el Establecimiento activo. No montar cinco instancias vivas simultaneas.
Selector de area	Territorio ofrece acceso claro a los Establecimientos disponibles. En el piloto se muestran La Manuela, El Triunfo, El Desafio, San Ignacio y El Rocio.
Encuadre inicial	Al seleccionar un Establecimiento, ajustar la camara a su geometria canonica con contexto alrededor. El margen exacto se define en UX-0b/UX-0c y no se hardcodea como regla de dominio.
Limites de navegacion	Aplicar limites de pan/zoom coherentes con el area seleccionada y su contexto. No permitir desplazamiento arbitrario a zonas sin relacion con el trabajo territorial.
Capas	Cargar la geometria canonica y las capas necesarias del Establecimiento activo. Las fuentes individuales pueden mostrarse como capa de procedencia cuando el flujo lo requiera.
Otras areas	Las areas no activas se representan con metadata, selector o previews livianos si UX lo decide; no necesitan mantener geometria detallada ni mapas activos en memoria.

G. Regla de rendimiento y memoria

EFICIENCIA	La interfaz no debe precargar simultaneamente todas las geometrias detalladas, capas y tiles de todos los Establecimientos cuando el usuario trabaja sobre uno solo. El fetch, render y cache deben priorizar el area activa y su contexto necesario.
------------	---

- La optimizacion es una regla de producto, no una promesa de consumo fijo de RAM: se mide y ajusta con datos reales.
- Restringir el viewport reduce navegacion irrelevante y evita acumulacion innecesaria de trabajo de render/tile cache, pero no sustituye profiling si aparece un problema real.
- No se cambia MapLibre ni se introduce un motor cartografico nuevo por esta necesidad.
- Si se usan miniaturas para las cinco areas, deben ser previews livianos/estaticos o equivalentes, no cinco mapas interactivos completos.

ACTUALIZACION v4.10 - API, UX-0 Y CIERRE DE ETAPA 3

H. Ajustes en API territorial y consultas

Superficie	Ajuste aditivo
Listado territorial	Debe poder obtener un resumen liviano de Establecimientos para selector/navegacion: identidad, nombre/codigo y metadata estrictamente necesaria. No obliga a devolver todas las geometrias detalladas.
Detalle por Establecimiento	Debe poder solicitar el Establecimiento seleccionado con su geometria canonica, LoteBase/contexto requerido y, cuando aplique, fuentes/procedencia. Reutilizar contratos existentes si ya lo permiten.
Fuentes	Las consultas tecnicas/operativas de fuentes siguen separadas de la vista canonica. El frontend no reconstruye la verdad territorial a partir de datos crudos si el backend ya posee la canonica autoritativa.
Cache/ETag	Puede incorporarse mas adelante si aporta valor medible; v4.10 no obliga a una estrategia prematura de cache distribuido.

I. UX-0c - Territorio como superficie de referencia

UX-0c debe aplicar DESIGN.md al modelo focalizado: selector de Establecimiento, un mapa activo, contexto alrededor, panel/ficha contextual, zoom/pan acotados, estados de carga/error/dato incompleto y accesibilidad. La experiencia debe comunicar claramente en que campo se esta trabajando y evitar que el usuario pierda contexto por navegacion cartografica ilimitada.

- Desktop: mapa y ficha pueden coexistir con alta densidad operativa.
- Tablet/celular: preservar seleccion, contexto y acciones principales sin intentar replicar el layout desktop a escala reducida.
- La navegacion y seleccion no dependen exclusivamente de color.
- Cambiar entre Establecimientos libera/reemplaza las fuentes visuales del mapa activo en lugar de acumular capas innecesarias.

J. Slice 3.7 - criterio de cierre territorial ampliado

CRITERIO DE SALIDA	Ademas de todos los criterios existentes, el piloto debe poder representar los cinco Establecimientos logicos confirmados, conservar sus fuentes individualmente, navegar cada area de forma independiente con contexto local y demostrar que correcciones/reagrupaciones no alteran silenciosamente Campanas/UOP historicas. La vista no requiere cargar los cinco mapas interactivos simultaneamente.
--------------------	---

ACTUALIZACION v4.10 - PRUEBAS, PILOTO E IMPACTO TRANSVERSAL

K. Pruebas adicionales

Nivel	Prueba esperada
DB/PostGIS	Reasignar una fuente entre Establecimientos conserva historia, mantiene una unica contribucion vigente intencional y recomputa canonicas correctas.
Geometria	La union de fuentes vigentes conserva MultiPolygon, huecos y componentes separados; no usa convex hull. Las fuentes originales siguen consultables.
Historia	Una correccion actual no cambia silenciosamente la representacion territorial historica requerida por Campanas/UOP anteriores.
Idempotencia	La misma solicitud de correccion/reagrupacion repetida produce una sola consecuencia.
API	El selector/listado no necesita transportar todas las geometrias detalladas; el detalle de un Establecimiento devuelve solamente el contexto requerido.
Frontend	Cambiar de area reemplaza el mapa/capas activas; no monta cinco mapas vivos. max bounds/limites equivalentes impiden navegacion global no intencional.
E2E	Territorio -> seleccionar uno de los cinco campos -> visualizar poligonos/contexto -> zoom/pan local -> cambiar de campo -> ficha y geometria correctas.
Performance	Verificar con datos piloto que la estrategia focalizada no introduce acumulacion evidente de instancias/capas/requests al alternar repetidamente entre campos.

L. Dataset piloto y decisiones pendientes

Tema	Estado v4.10
Cinco Establecimientos	CONFIRMADO como estructura operativa del piloto: La Manuela, El Triunfo, El Desafio, San Ignacio y El Rocio. No hardcodear.
Ceibo	CONFIRMADO por negocio como parte de San Ignacio; preservar nombre/evidencia externa.
"La Manuela" aislada	PENDIENTE: revisar coordenadas del registro concreto y confirmar si pertenece a San Ignacio.
Resto de agrupaciones	VALIDAR con coordenadas y confirmacion de negocio antes de cerrar el dataset definitivo.
Margen visual alrededor	PENDIENTE DE UX-0b/UX-0c: definir regla de encuadre/contexto y limites de navegacion con datos reales.
Solapamiento LoteBase	SIGUE PENDIENTE: v4.10 no modifica la decision funcional/tolerancias de v4.7-v4.9.

ACTUALIZACION v4.10 - SECCIONES AFECTADAS Y PRIORIDAD

M. Secciones del Plan Integral ampliadas

Seccion previa	Ajuste v4.10
2 - Principios	Se agrega navegacion territorial focalizada y agrupacion por evidencia espacial + confirmacion de negocio, sin alterar ownership ni no-delete.
3 - Arquitectura	MapLibre sigue siendo la superficie cartografica. Se prioriza una instancia/mapa activo y carga por Establecimiento; no aparecen nuevos servicios.
5 - CI/pruebas	Se agregan regresiones de reagrupacion/versionado y checks de no acumulacion evidente de mapas/capas en el flujo focalizado.
6 - DoD	Un flujo territorial visible debe operar por contexto de Establecimiento, cubrir estados y no exigir navegacion global o carga simultanea innecesaria.
10 - Etapa 3 / 3.4	API debe soportar listado liviano y detalle focalizado cuando no exista ya un contrato equivalente.
10 - Etapa 3 / 3.5	Mapa se interpreta como vista del Establecimiento activo; la experiencia visual final se consolida en UX-0c.
10 - Etapa 3 / 3.6e	Correccion/versionado incluye reasignar una fuente entre Establecimientos, recomputar canonicas y preservar historia.
UX-0c	Implementar selector de area, mapa unico activo, fit/limites de navegacion, contexto local, responsive y accesibilidad.
3.7	Validar los cinco Establecimientos del piloto, agrupaciones confirmadas, historia y navegacion independiente.
21 - Piloto/cutover	Inventario territorial del piloto incluye agrupacion final de fuentes y procedimiento para corregirla sin destruir evidencia.
23 - Matriz pruebas	Territorio agrega versionado/reagrupacion, API focalizada, E2E por area y performance de cambio entre campos.
25 - Pendientes	Mantener explicitamente las agrupaciones aun no confirmadas y la politica de solapamiento LoteBase como decisiones pendientes.
26 - Roadmap	No cambia: 3.6e -> UX-0a -> UX-0b -> UX-0c -> 3.7 -> Etapa 4. v4.10 no crea etapa nueva.

REGLA DE PRIORIDAD

v4.10 es estrictamente aditiva. No elimina, sustituye, posterga ni reduce autenticacion, autorizacion, Panel Administrativo, Internal Console, audit, jobs, outbox, Documents, idempotencia, Territorio/SENASA/RENSPA, Stock, Agricultura, Maquinaria, Ganaderia/XRS2i, Granos, Comercializacion, ARCA, Finnegans, Costos/MB, Conciliacion, Piloto, staging, CI, HTTP/2+, seguridad, no-delete, Decimal, SOLID, alta cohesion, bajo acoplamiento, modularidad, abstraccion, UX/UI, accesibilidad ni ningun criterio de salida anterior.